

Reconnaître des caractères dans des monogrammes de sceaux byzantins : un pipeline dannotation semi-automatique

Mémoire

présenté par

Mamadou Diouhé BARRY

Ezéchiel SAWADOGO

Master 1
Mention « *Sciences du Langage* »
Parcours « *Langue et Informatique* »

dirigé par

Victoria EYHARABIDE

soutenu le
04 Juin 2026

Que la force me soit donnée de supporter ce qui ne peut être changé,
le courage de changer ce qui peut l'être,
et la sagesse de distinguer l'un de l'autre.

MARC-AURÈLE

Dédicace

À nos familles, amis qui n'ont cessé de nous encourager,
qui ont toujours fait preuve de mansuétude à notre égard,
et nous ont relevé la tête quand nous semblions acculés.

À nos enseignants sans qui nous ne serions pas là,
et qui ont su par leurs mots nous transmettre,
à travers leurs cours, la passion des études.

Et à ceux qui, après nous,
continueront de chercher dans le passé
ce qui éclairera le présent.

Nous dédions ce travail au courage, à l'obstination
et à la volonté qui nous ont accompagnés
tout au long de cette traversée.

Remerciements

Nous ne pourrions débuter cet ouvrage sans adresser notre profonde gratitude à Madame Victoria EYHARABIDE, notre encadrante de mémoire, pour sa disponibilité, ses conseils avisés et la confiance qu'elle nous a accordée tout au long de ce travail. Nos remerciements vont également à Monsieur Saranga K. MAHANTA pour son accompagnement et ses orientations précieuses.

Nous tenons à remercier chaleureusement Madame Béatrice CASEAU pour nous avoir permis d'assister à sa conférence, dont les éclairages historiques ont enrichi notre compréhension du corpus sigillographique byzantin.

Nos remerciements vont aussi à l'équipe pédagogique du Master Langue et Informatique de la Sorbonne Université pour l'accompagnement et la qualité des enseignements dispensés, ainsi qu'aux enseignants qui ont bien voulu consacrer de leur temps à la relecture de ce manuscrit. Nous remercions enfin l'ensemble des chercheurs du projet BHAI, dont les travaux ont nourri notre réflexion.

Résumé

Ce mémoire, intitulé *Reconnaître des caractères dans des monogrammes de sceaux byzantins : un pipeline d'annotation semi-automatique*, se situe à l'intersection de la sigillographie (l'étude des sceaux anciens) et de la vision par ordinateur. Face à la richesse du patrimoine byzantin et à la raréfaction de l'expertise humaine, les auteurs proposent un pipeline semi-automatique pour assister la reconnaissance des caractères grecs gravés sur des sceaux en plomb.

La méthodologie repose sur deux axes principaux. D'une part, la détection des caractères est effectuée par un modèle YOLOv8s (You Only Look Once), entraîné sur un corpus de 351 images annotées manuellement sur la plateforme Supervisely, selon un protocole de validation croisée à 5 plis. D'autre part, une étude comparative pour la classification des caractères extraits oppose un CNN entraîné *from scratch*, un ResNet18 en *transfer learning*, et un Vision Transformer (ViT-Tiny).

Les résultats montrent que YOLOv8s atteint une mAP@50 moyenne de 0,778, démontrant sa viabilité pour localiser des caractères sur des supports dégradés. Pour la classification, ResNet18 et ViT-Tiny atteignent tous deux une accuracy de 96,66%, mais ResNet18 se distingue sur les métriques macro (précision (0,933), rappel (0,942) et F1-score (0,937)) traduisant une meilleure robustesse sur les classes rares. L'analyse des erreurs met en lumière les principaux défis : le déséquilibre des classes (caractères rares), les ambiguïtés morphologiques (confusions entre lettres similaires) et les limites des boîtes englobantes face aux monogrammes entrelacés. Le mémoire conclut sur l'importance de l'humain dans la boucle et propose des perspectives telles que l'augmentation des données, la segmentation d'instance, l'intégration d'un modèle de langue pour assister le déchiffrement et l'élaboration d'un schéma de qualification de la qualité des images (M0 à M3), ouvrant la voie à un filtrage automatique en amont du pipeline.

Mots-clés : – Sigillographie byzantine
– Détection d'objets
– YOLOv8
– Classification de caractères
– Transfer learning
– ResNet18
– Vision Transformer (ViT)
– Humanités numériques
– Pipeline semi-automatique
– Annotation manuelle

Liste des abréviations

CNN	Convolutional Neural Network
COCO	Common Objects in Context
DFL	Distribution Focal Loss
GPU	Graphics Processing Unit
HMM	Hidden Markov Models
HOG	Histogram of Oriented Gradients
HPC	High-Performance Computing
HTR	Handwritten Text Recognition
IoU	Intersection over Union
JSON	JavaScript Object Notation
mAP	mean Average Precision
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
PANet	Path Aggregation Network
R-CNN	Region-based Convolutional Neural Network
ResNet	Residual Network
SSD	Single Shot MultiBox Detector
SVM	Support Vector Machine
TEI	Text Encoding Initiative
ViT	Vision Transformer
XML	eXtensible Markup Language
YOLO	You Only Look Once
BHAI	Byzantine Hybrid Articial Intelligence

Table des matières

Résumé iii

Liste des abréviations v

Table des figures 4

Introduction Générale 6

I. L'écriture et le besoin d'authentification . . . 7

1.1 Les premiers supports . . . 8

1.2 Du papyrus au parchemin . . . 8

1.3 Protéger et authentifier : une nécessité universelle . . . 8

1.4 Les premiers sceaux : de Sumer à Rome . . . 9

II. Les sceaux dans l'Empire byzantin . . . 10

2.1 L'héritage romain adapté . . . 10

2.2 Iconographie et inscription : une double lecture . . . 11

III. Problématique : des sceaux au numérique . . . 11

1. Une annotation nécessaire mais exigeante . . . 12

2. Ambition computationnelle et réalité institutionnelle . . . 13

3. Des choix techniques ouverts . . . 13

IV. Organisation du mémoire . . . 14

CHAPITRE I : État de l'art : Vision par ordinateur et patrimoine écrit, approches sur les sceaux byzantins 16

. . . 16

I. La sigillographie face au traitement numérique . . . 16

1. Nature et diversité des sources historiques . . . 16

2.Variabilité paléographique . . . 18

II. Des modèles classiques à la reconnaissance de documents historiques . . . 18

1. Les premières approches . . . 18

2. Passage aux méthodes statistiques . . . 18

III.L'émergence du Deep Learning en vision par ordinateur 19
1.Rupture méthodologique 19
2.Applications au patrimoine culturel 20
IV.Détection d'objets : une reformulation nécessaire pour les sceaux byzantins 21
1. Les architectures modernes de détection d'objets 21
2. La famille YOLO : principe, évolution et pertinence pour notre corpus 23
3. Travaux appliqués aux sceaux byzantins 24
. 25

CHAPITRE II : Méthodologie : Un pipeline semi-automatique **28**
I. Détection automatique des caractères avec YOLO 28
1. Constitution du corpus annoté 28
2. Choix de la version et configuration du modèle 29
3. Stratégie d'entraînement et validation croisée 30
4. Production automatique de bounding boxes 31
II. Étude comparative des architectures de classification 32
1. Architectures retenues (CNN, ResNet18, ViT-Tiny) 32
2. Prétraitement et construction du jeu de données 33
3. Protocole d'évaluation 33

CHAPITRE III : Expérimentations et résultats **36**
I. Résultats quantitatifs de la détection 36
1. Résultats par fold 36
2. Courbes et visualisations 37
2.1 Courbes d'entraînement 37
2.2 Courbes Precision-Recall 39
2.3 Courbes de confiance F1 40
3. Matrice de confusion et analyse par classe 42
II. Résultats qualitatifs de la détection 44
III. Limites observées et analyse des erreurs 47
IV. Synthèse des résultats 48

CHAPITRE IV : Discussion **51**
I. Apports du pipeline proposé 51
II. Classification des caractères : étude comparative 52
1. Configuration générale 53
A. Modèle 1 : CNN entraîné from scratch 53
B. Modèle 2 : ResNet18 (transfer learning) 53

C. Modèle 3 : ViT-Tiny 54
2. Résultats quantitatifs (CNN, ResNet18, ViT-Tiny) 55
3. Courbes d'apprentissage 55
4. Analyse par classe et cas problématiques 57
III. Perspectives d'amélioration 58

Conclusion Générale **61**

Annexe A : Exemples dimages annotées **64**

Annexe B : Tableau des classes de caractères **67**
B.1 Classes de détection YOLOv8s 67
B.2 Classes de classification 67

Bibliographie **69**

Table des figures

1 Exemple de sceau byzantin en plomb présentant un monogramme cruciforme (Collection Seyrig 019) . . . 7
2 Sceau-cylindre mésopotamien et son empreinte sur argile. Le cylindre de pierre gravé (à gauche) est roulé sur l'argile molle pour produire la scène en relief visible sur la tablette (à droite). . . . 9
3 Scarabée ailé égyptien en faïence bleue turquoise. Symbole du dieu solaire Khépri, cet objet servait à la fois d'amulette protectrice et de sceau d'authentification. 10
4 Sceau byzantin fortement dégradé (Tatish 2318) : oxydation, usure mécanique et lacunes de matière illustrent les altérations que les algorithmes doivent apprendre à distinguer des traits d'écriture intentionnels. . . . 17
5 Timeline extrait de l'étude présentée par zou. . . . 22
6 Illustrations de la méthode *Viola-Jones* montrant la sélection des features proéminents. . . . 23
7 Exemple d'annotations au format YOLO. . . . 29
8 Courbes d'entraînement et métriques de validation, fold 4 (100 époques) . . . 37
9 Courbes d'entraînement et métriques de validation, fold 2 (100 époques) . . . 38
10 Courbe Precision-Recall, fold 4 (mAP@0,5 = 0,856) . . . 39
11 Courbe Precision-Recall, fold 2 (mAP@0,5 = 0,720) . . . 39
12 Courbe F1-Confidence, fold 4 (F1 = 0,78 au seuil optimal de 0,439) . . . 40
13 Courbe F1-Confidence, fold 2 (F1 = 0,67 au seuil optimal de 0,327) . . . 41
14 Matrice de confusion normalisée, fold 4 . . . 42
15 Matrice de confusion normalisée, fold 2 . . . 43
16 Comparaison entre les annotations de référence et les détections automatiques YOLOv8s, Fold 4 . . . 44
17 Comparaison entre les annotations de référence et les détections automatiques YOLOv8s, Fold 2 . . . 45
18 Distribution des classes, fold4 . . . 46
19 Exemple de deux ou plusieurs lettres qui partagent une même région verticale. . . 48
20 courbes Test Loss et Test Accuracy CNN vs ResNet18 vs ViT-Tiny . . . 56

21 F1-Score par classe pour les 26 classes (3 modèles) 57
22 Monogramme annoté 268 (Supervisely) 64
23 Monogramme annoté 270 ? (Supervisely) 64
24 Monogramme annoté 271 ? (Supervisely) 65
25 Monogramme annoté 272 (Supervisely) 65

Introduction Générale

De tous temps et en tous lieux, les Hommes ont toujours eu ce besoin impératif de fixer leurs pensées, leurs accords et leurs transactions sur des supports fiables et durables. Toutefois, l'écriture ne saurait être la seule compétence requise pour satisfaire ce besoin. Il faut pouvoir authentifier le message, prouver son origine et protéger son contenu des regards indiscrets et des falsifications. Cette double exigence (conserver et authentifier le message) traverse l'histoire de l'Humanité comme un fil conducteur, des premières tablettes sumériennes aux signatures numériques de notre époque.

Les sceaux byzantins, objets de notre étude, s'inscrivent pleinement dans cette tradition. Ces petits objets, parfois d'or, parfois de plomb ou d'argent, représentent l'aboutissement de plusieurs millénaires d'évolution des pratiques liées à l'authentification documentaire. Ils témoignent de l'avancement administratif d'un empire qui a subsisté durant mille ans, s'adaptant aux bouleversements sans jamais faillir à transmettre l'héritage des civilisations antérieures jusqu'à sa chute en 1453.

Malgré la richesse que constitue ce corpus, l'étude systématique des caractères qui ornent ces sceaux reste encore largement tributaire du travail manuel des spécialistes en sigillographie. Ceux-ci, dont le nombre se raréfie et dont le savoir-faire, aussi précieux soit-il, ne peut à lui seul absorber les dizaines de milliers de pièces encore à étudier.



FIGURE 1 – Exemple de sceau byzantin en plomb présentant un monogramme cruciforme (Collection Seyrig 019)

Notre travail de recherche se situe précisément à l'intersection entre ce patrimoine et les technologies contemporaines. Les modèles d'apprentissage profond (Deep Learning) étant **affamés** de données structurées, il s'agir de déterminer comment mobiliser les avancées récentes des modèles d'apprentissage, et notamment de l'apprentissage profond, afin d'assister les sigillographes dans la reconnaissance des caractères présents sur les sceaux byzantins. Telle est l'une des nombreuses questions qui portera ce mémoire et auxquelles il se devra de répondre. Ce postulat oriente cette réflexion intitulée « Reconnaître des caractères dans des monogrammes de sceaux byzantins : un pipeline d'annotation semi-automatique ».

Avant de présenter notre démarche méthodologique et nos contributions, il nous paraît important de revenir brièvement sur le contexte historique dans lequel s'inscrit notre objet de réflexion. Cette introduction propose donc une synthèse de l'histoire des supports d'écriture et des diverses pratiques d'authentification qui ont pu se succéder, répondant à la double exigence déjà mentionnée. Ce postulat posé, le lecteur pourra comprendre que nous aborderons le rôle central des sceaux dans l'empire, puis les défis contemporains que pose leur étude numérique.

I. L'écriture et le besoin d'authentification

Sans prétendre ici à un exposé exhaustif de l'histoire de l'écriture, que Madame Béatrice Caseau, directrice de l'UFR d'histoire à la Sorbonne Université a remarquablement retracée, lors d'une conférence donnée le mardi 03 février 2026. Il conviendra alors de rappeler brièvement les étapes essentielles qui permettent de comprendre comment l'on est arrivé aux bulles de plomb de l'Empire byzantin.

1.1 Les premiers supports

Selon les historiens, l'écriture est apparue vers 3300 avant notre ère en Mésopotamie (actuelle Irak). Cette apparition marque une rupture inéluctable dans l'histoire de l'Humanité. Pour la première fois, l'on pouvait observer les formes physiques mais conceptuelles des productions sonores. Pour la première fois, la parole était fixée, gravée sur un support. Elle pouvait être conservée, mieux encore, elle était transmissible. Cette révolution nécessitait des supports adaptés ; ainsi naquirent les premiers documents.

Dans l'ancienne Mésopotamie, berceau de l'écriture cunéiforme, le support choisi était une tablette d'argile. Ce choix s'imposait naturellement : abondante dans les plaines boueuses du Tigre et de l'Euphrate, l'argile est facile à modeler et durcit en séchant. Pour les documents importants, on la cuisait au four, ce qui avait pour effet de la rendre encore plus solide. Plusieurs d'entre elles ont été retrouvées, témoignant de l'activité administrative des civilisations sumériennes, akkadiennes et babyloniennes.

Parallèlement, plus loin en Égypte, se développe un autre support majeur de l'histoire : le papyrus. Fabriqué à partir de tiges d'une plante poussant en abondance près du Nil, le papyrus présente des qualités remarquables. Léger, souple, sa surface lisse se prête facilement à l'écriture au pinceau. Dès le début du IIIe millénaire, on assiste à un usage intensif de ce support tant pour l'administration que pour les textes sacrés.

1.2 Du papyrus au parchemin

Le papyrus connaît un succès immense dans l'ensemble du monde méditerranéen. Les Grecs l'adoptent au VIe siècle av J.-C., suivis par les Romains au IIIe siècle av J.-C. L'Égypte devient alors le fournisseur de ce matériau, ce qui confère à l'empire un avantage économique et stratégique considérable.

Néanmoins, bien que le papyrus ait de nombreux avantages, celui-ci comporte son lot d'inconvénients notables. Il est fragile, craint l'humidité et sa production dépend d'une seule zone du monde (l'Égypte). C'est dans ce contexte que naît une alternative : le parchemin. Selon les historiens, c'est à Pergame (localisation actuelle Turquie) que se développe et se perfectionne la technique qui consiste à préparer des peaux d'animaux (mouton, veau, chèvre) pour en faire un support. Le nom même de « parchemin »(*pergamena* en latin) rappelle cette origine.

1.3 Protéger et authentifier : une nécessité universelle

Disposer d'un support d'écriture ne saurait suffire, comme nous l'avons mentionné plus haut. Dès qu'un document revêt une valeur juridique ou autre, se pose la question de son authenticité. Comment prouver qu'un contrat n'a pas été falsifié ? Comment attester que l'ordre émis émane bien d'une autorité légitime ? Comment s'assurer que le contenu du message n'a

pas été altéré par un tiers ? Autant de questions qui nous animent encore aujourd'hui à l'ère du numérique. Parmi les solutions apportées durant l'Antiquité, on peut distinguer :

- la **signature**, marque personnelle de l'émetteur, pouvant prendre la forme d'un nom ou d'un symbole ;
- les **témoins**, personnes présentes lors de la rédaction du document ;
- le **sceau**, empreinte apposée à un document à base d'argile, de cire, de plomb, d'or ou d'argent, qui authentifie l'origine du document et en garantit l'inviolabilité.

À travers ce qui est présenté ici, nous pouvons observer que le sceau possède une particularité remarquable par rapport aux autres dispositifs d'authentification. Apposé à un document, il empêche toute infraction ou falsification sans rupture visible, constituant ainsi l'une des premières formes de sécurité « inviolable »de l'histoire documentaire.

1.4 Les premiers sceaux : de Sumer à Rome

C'est en Mésopotamie qu'apparaissent les premiers sceaux connus. Il s'agit de petits cylindres de pierre gravés en creux, que l'on roule sur de l'argile encore molle pour y laisser une empreinte en relief. Ces sceaux-cylindres représentent la plupart du temps des scènes mythologiques, des animaux ou des motifs géométriques. Objets personnels portés au cou ou au poignet, ils servaient à attester l'authenticité des contrats commerciaux, à sceller les jarres de denrées, à marquer la propriété. Chaque sceau constituait une véritable carte d'identité de son possesseur.



FIGURE 2 – Sceau-cylindre mésopotamien et son empreinte sur argile. Le cylindre de pierre gravé (à gauche) est roulé sur l'argile molle pour produire la scène en relief visible sur la tablette (à droite).

En Égypte, le sceau prend une forme toute autre : celle d'un scarabée. Insecte associé à la divinité solaire Khépri et symbole de renaissance, sculpté dans la pierre, il servait à la fois d'amulette protectrice et d'instrument d'authentification dans l'empire. Les Grecs, quant à eux, adoptent le sceau sous une autre forme : la bague à chaton gravé, ornée d'une pierre précieuse ou semi-précieuse. Au fil du temps, l'usage se répand dans toutes les couches de la société.



FIGURE 3 – Scarabée ailé égyptien en faïence bleue turquoise. Symbole du dieu solaire Khépri, cet objet servait à la fois d'amulette protectrice et de sceau d'authentification.

Les Romains perfectionnent le système. L'anneau sigillaire (*annulus signatorius*) devient un attribut courant des citoyens romains. Pour les documents officiels, on développe l'usage des *bullae*, masses de cire ou de métal attachées au document par un cordon. Les empereurs utilisent des sceaux de plomb pour leurs actes les plus solennels, une pratique dont héritera l'Empire byzantin.

II. Les sceaux dans l'Empire byzantin

1. L'héritage romain adapté

C'est à partir du VIe siècle apr. J.-C. que l'usage des sceaux se développe à Byzance de façon systématique. Réservées dans un premier temps aux empereurs, les bulles de plomb se démocratisent progressivement pour s'étendre à l'ensemble de l'appareil administratif : stratèges, fonctionnaires fiscaux, patriarches, évêques. Entre 70 000 et 80 000 bulles de plomb ont été retrouvées à ce jour, datant pour la plupart du VIe au XIIe siècle, auxquelles s'ajoutent quelques dizaines de bulles d'or réservées aux actes les plus importants.

Ces objets sont généralement bifaces : une face iconographique (effigie de la Vierge, du

Christ, d'un saint patron) et une face épigraphique portant l'identité du sigillant. Le contenu de ces inscriptions s'enrichit au fil des siècles : au VIe siècle, un simple prénom ; à partir du VIIIe siècle, la dignité et la fonction ; au XIe siècle, les bulles les plus informatives combinent nom de baptême, dignités, fonctions, lieu d'exercice et parfois le nom de famille. La forme même des lettres évolue : à partir du XIIe siècle, les monogrammes adoptent une structure fondée sur une barre verticale centrale permettant de condenser fortement l'écriture. Ces évolutions stylistiques constituent des indices précieux pour dater et identifier les sceaux, mais elles constituent aussi, comme nous allons le voir, l'une des principales difficultés de leur traitement automatique.

Authenticité et intégrité : des concepts qui ne nous sont pas inconnus. En cryptographie moderne, la signature numérique remplit exactement cette double fonction. Le sceau byzantin n'est ainsi que l'ancêtre de nos protocoles de sécurité contemporains.

2. Iconographie et inscription : une double lecture

Comme nous l'avons abordé plus haut, les sceaux sont généralement des objets bifaces qui portent de l'iconographie et toujours du texte. Les enseignements qui nous sont fournis par les sceaux byzantins sont de deux ordres, suivant qu'ils proviennent de l'étude des types ou de celle des légendes (inscriptions textuelles). « L'étude des types gravés sur les sceaux est une source d'indications précieuses, surtout pour la connaissance de l'iconographie religieuse byzantine »[18]. Sur un très grand nombre de sceaux figurent les effigies de la Vierge, du Christ, des saints, dans les attitudes, sous les traits, avec le costume et les attributs traditionnels particuliers à chacun.

III. Problématique : des sceaux au numérique

La richesse des sceaux byzantins, bien que reconnue, se heurte visiblement à un obstacle structurel que les méthodes traditionnelles ne permettent pas de surmonter ou de traiter de manière *ad hoc*. Les milliers de sceaux répertoriés aujourd'hui ont certes bénéficié de publications savantes accompagnées de descriptions minutieuses ; en témoignent les nombreux travaux d'auteurs tels que Gustave Schlumberger (1884), le Père Vitalien Laurent (1963), Georges Zacos, Alexander Veglery (1972) et Jean-Claude Cheynet constituant un *ground truth* textuel d'une fiabilité indéniable. Néanmoins, ce savoir reste confiné au support imprimé. C'est ce constat qui pousse Eyharabide (2024)[5] à souligner qu'à l'exception de la base de données de Dumbarton Oaks, l'ensemble des collections de sceaux byzantins demeure uniquement accessible sous forme de publication imprimée, et qu'il n'existe à ce jour aucune base de données unifiée, interopérable et annotée pouvant servir de *ground truth* pour l'entraînement de modèles.

Or, les sceaux souffrent d'un état de conservation souvent dégradé : corrosion, oxydation, cassures, usure rendent la lecture des inscriptions particulièrement ardue, y compris pour les

spécialistes. Les caractères gravés sur ces petites pastilles de plomb sont fréquemment abîmés, parfois partiellement effacés, et leur identification requiert une expertise considérable. L'annotation de ces caractères (étape indispensable pour constituer des bases de données d'entraînement) est un processus extrêmement chronophage et coûteux, qui ne peut être réalisé qu'à une échelle limitée (tel est notre cas ici).

Face à ces contraintes techniques, les avancées récentes en vision par ordinateur et en apprentissage profond ouvrent des voies prometteuses. Des travaux menés dans le cadre du projet BHAI (Byzantine Hybrid Artificial Intelligence) ont posé la possibilité d'appliquer des réseaux de neurones profonds à la reconnaissance de caractères sur des sceaux byzantins[6, 13]. Par ailleurs, des méthodes de caractérisation automatique de la dégradation des bords et de délimitation du contenu ont été développées en parallèle, ouvrant la voie à un traitement plus systématique des images.

C'est dans cet élan scientifique que s'inscrit le présent mémoire, dont la problématique centrale peut être formulée comme suit :

> **Comment concevoir un pipeline semi-automatique permettant de détecter et de classifier les caractères sur les images de sceaux byzantins, puis d'aligner ces détections avec les transcriptions publiées dans les catalogues sigillographiques, en vue de constituer une base d'images annotées exploitable pour l'entraînement de modèles de reconnaissance ?**

Cette question en appelle plusieurs autres : quel modèle de détection d'objets convient le mieux à la localisation de caractères sur des images dégradées ? Quel protocole de classification faut-il utiliser ? Comment extraire et structurer les transcriptions cataloguées pour les rendre opérationnelles grâce à l'outil computationnel ? Quel mécanisme d'alignement permet de mettre en correspondance les zones détectées sur l'image avec les caractères transcrits ? Et selon quels critères évaluer la qualité du pipeline ainsi conçu par rapport à une annotation entièrement manuelle ?

Telles sont les questions qui structurent notre démarche et auxquelles ce mémoire s'efforcera d'apporter des éléments de réponse.

1. Une annotation nécessaire mais exigeante

Pour initier ce pipeline, nous avons opté pour une phase d'annotation manuelle sur la plateforme Supervisely. Ce choix n'est pas anodin : il est le résultat d'une réflexion méthodologique sur ce que le modèle doit apprendre à « voir ». Le travail d'annotation consiste à délimiter manuellement chaque caractère par une *bounding box*, puis à lui assigner une classe parmi un ensemble standardisé (lettres, symboles...) afin d'assurer la cohérence des données entre les différents annotateurs. En exportant ces annotations au format JSON, nous permettons au modèle

YOLO d'accéder à une représentation spatiale fine de chaque caractère, avec ses coordonnées précises dans l'image. Ce niveau de finesse nous a paru indispensable pour une détection robuste sur des objets aussi petits et dégradés que les caractères gravés sur des bulles de plomb.

Ce processus a cependant rapidement mis en lumière des difficultés que seule la pratique concrète de l'annotation révèle. Les caractères byzantins ne se présentent pas toujours comme des entités discrètes et isolées : ils s'agglutinent, se superposent, se fondent les uns dans les autres. Face à une forme donnée, l'annotateur doit trancher des questions qui ne sont pas triviales : s'agit-il d'un epsilon isolé ou d'une ligature ? Cet alpha est-il indépendant, ou est-il fusionné avec un delta voisin ? Deux caractères ont-ils été gravés conjointement au point de ne former visuellement qu'un seul signe ? Ces débats de labellisation, qui peuvent sembler anodins, conditionnent directement la qualité du *ground truth* et, par conséquent, les performances du modèle. Ils illustrent la nécessité d'un protocole d'annotation rigoureux et d'une expertise sigillographique que l'outil informatique seul ne peut pas fournir.

2. Ambition computationnelle et réalité institutionnelle

Si l'annotation constitue le point de départ indispensable, le coeur technique de ce projet réside dans le déploiement de modèles de détection sur des infrastructures de calcul intensif. L'entraînement de réseaux de neurones profonds nécessite en effet des ressources GPU considérables pour traiter les images en parallèle, et la recherche de la configuration optimale du modèle (choix de la version de YOLO, réglage des hyperparamètres...) demande des centaines d'expérimentations qu'un ordinateur personnel ne peut absorber dans un délai raisonnable.

C'est dans cette perspective que nous avons sollicité l'accès aux ressources du cluster SACADO (Plateforme MeSU) de la Sorbonne Université. Cependant, en raison de protocoles de sécurité stricts et d'une priorité accordée aux financements de recherche spécifiques, l'accès à ces ressources est rarement ouvert aux étudiants de Master 1. Notre demande n'ayant pas reçu d'issue favorable, nous avons dû adapter notre stratégie technique. Une contrainte dont nous discutons les implications dans ce mémoire.

3. Des choix techniques ouverts

Ce travail soulève également des questions techniques dont certaines restent ouvertes au moment de la rédaction. Parmi elles, le choix de la version de YOLO à déployer pour optimiser les résultats sur ce type d'images est une décision qui sera argumentée dans le chapitre lié à la méthodologie.

IV. Organisation du mémoire

Pour répondre à cette problématique, ce mémoire s'organise en quatre chapitres :

- Le **premier chapitre** sera consacré à un état de l'art des méthodes de détection et de reconnaissance de caractères sur des documents anciens, avec une attention particulière aux travaux appliqués aux sceaux byzantins. Nous y présenterons également les architectures de détection d'objets pertinentes, notamment les modèles de la famille YOLO.
- Le **deuxième chapitre** décrira notre méthodologie : la constitution du jeu de données, le processus d'annotation sur la plateforme Supervisely, et l'entraînement des modèles de détection.
- Le **troisième chapitre** présentera les résultats expérimentaux : performances du modèle de détection YOLOv8s ainsi que leur évaluation. Puis nous discuterons des forces et des limites du pipeline proposé.
- Le **quatrième chapitre**, dédié à la discussion, approfondira les apports du pipeline, présentera l'étude comparative des trois architectures de classification (CNN, ResNet18, ViT-Tiny) et proposera des perspectives d'amélioration.
- Enfin, nous **conclurons** en synthétisant nos contributions, en identifiant les perspectives d'amélioration et en évoquant les possibilités de publication et de prolongement de ce travail dans le cadre d'autres projets.

CHAPITRE I

État de l'art : Vision par ordinateur et patrimoine écrit, approches sur les sceaux byzantins

Comme nous lavons evoqué dans lintroduction générale, létude des sceaux byzantins se situe à la croisée de deux domaines traditionnellement distincts : dune part les sciences historiques (la sigillographie : discipline consacrée à létude des sceaux anciens) ; dautre part les sciences informatiques contemporaines, notamment la vision par ordinateur et lapprentissage profond. Lobjectif de ce chapitre est de présenter létat actuel des connaissances scientifiques relatives à la reconnaissance automatique dinscriptions. Il sagit moins d'effectuer un inventaire exhaustif des travaux existants que de mettre en évidence les approches méthodologiques ayant permis le passage progressif dune analyse documentaire manuelle vers des traitements computationnels automatisés. Dans cette perspective, trois axes seront successivement abordés. Nous examinerons tout dabord les spécificités des documents historiques et les difficultés propres à leur traitement numérique. Nous présenterons ensuite lévolution des méthodes de reconnaissance de caractères, depuis les approches classiques d'OCR jusquaux modèles fondés sur lapprentissage profond. Enfin, une attention particulière sera portée aux architectures modernes de détection dobjets, et notamment à la famille des modèles YOLO, aujourdhui largement utilisée pour la localisation déléments visuels complexes dans des images dégradées.

I. La sigillographie face au traitement numérique

1. Nature et diversité des sources historiques

Pour comprendre pourquoi les modèles neurenaux peinent encore à déchiffrer le passé, nous devons d'abord accepter une réalité : le sceaux est un objet « rebelle». Contrairement aux textes modernes, imprimés selon des normes stricts et parfaitement alignés, la source historique est intrinsèquement imprévisible. Nous nous appuyons ici sur le concept de Frédic Barbier qui évoque une « morphologie instable» [2] : c'est la nature même du support (qu'il soit d'argile,de papyrus ou de plomb) qui impose sa loi à l'écriture et dicte la forme finale de l'inscription. Dans ce panorama, les sceaux byzantins que nous étudions occupent une place singulière. Comme le démontre Jean-Claude Cheynet [4], un sceau n'est pas une surface plane comme une page de manuscrit, mais un véritable objet en trois dimensions. Les inscriptions y sont frappées en relief ou en creux sur un flan métalinque souvent irrégulier, bombé ou déformé par le temps. Pour les algorithmes peu habitués à ce genre de données ceci constitue un véritable choc épistémologique. Nous avons identifié que cette nature tridimensionnelle introduit trois obstacles critiques pour nos modèles de détection :

. Le piège de la lumière : Le plomb ou l'or renvoient la lumière de manière changeante. Selon l'angle de prise de vue, une ombre portée peut occulter un trait essentiel ou, au contraire,

crée un relief artificiel. Les traveaux de Malzbender sur l'imagerie computationnelle ont prové qu'une simple photographie en 2D est une interprétation « figée » de la lumière [11].qui peut tromper les algorithmes de reconnaissances de formes.

. L'anamorphose des formes : La surface d'un sceau est rarement plane. Sur les bords arrondis, les lettres subissent une distorsion géométrique : elles s'ecrasent ou s'étirent. Nous pouvons en conclure que ces courbures modifient les proportions originelles des caractères. Si notre modèle cherche un « Alpha » idéal mais rencontre une forme tordue par la sphéricité du plomb, le risque de non détection augmente.

. Le bruit de l'histoire : En l'espace de dix siècles , le metal a subi l'oxydation, des chocs ou l'usure mécanique. Les interruptions visuelles sur le sceau ne sont pas toujours des espaces volontaires entre les mots, mais des«morsures» du temps. Le défi est d'apprendre à nos algorithmes à distinguer une lettre partiellement effacée d'une simple rayure ou d'une lacune de matière.



FIGURE 4 – Sceau byzantin fortement dégradé (Tatish 2318) : oxydation, usure mécanique et lacunes de matière illustrent les altérations que les algorithmes doivent apprendre à distinguer des traits d'écriture intentionnels.

En résumé, un même caractère peut revêtir des apparences totalement hétérogènes d'une pièce à l'autre. C'est pourquoi, en suivant la direction tracée par Alessio Sopracasa (université sorbonne) et Martina Filosa (université de Cologne) avec le projet SigiDoc (projet d'encodage XML/TEI), nous considérons qu'il faut enrichir encore davantage les informations scientifiques exploitables en transformant ainsi ces contraintes materielles en informations scientifiques exploitables

2.Variabilité paléographique

Au-delà des altérations physiques du plomb, nous nous heurtons à un obstacle graphique : l'extrême instabilité des caractères sur le long terme. L'écriture byzantine est une matière vivante. Les différentes mutations des caractères que nous avons évoquées plus haut reflètent l'évolution stylistique des ateliers de gravure.

Cette difficulté est accentuée par la gestion de l'espace sur le sceau. Le graveur, prisonnier de la petite surface circulaire, doit souvent improviser. Nous observons un usage massif d'abréviations et de ligature complexes. Ces ligatures brisent la silhouette classique des lettres. Celles-ci deviennent presque méconnaissables pour un algorithme classique. un même signe peut condenser plusieurs lettres, défiant toute logique de segmentation simple. Enfin, l'habileté variable des graveurs, fait que chaque sceau est une pièce unique.

Notre pipeline doit donc développer une capacité d'abstration, permettant de reconnaître l'identité d'une lettre derrière la diversité de ses exécutions.

II. Des modèles classiques à la reconnaissance de documents historiques

1. Les premières approches

Pour bien mesurer le saut technologique que nous tentons de réaliser, il faut d'abord revenir aux racines de la reconnaissance optique de caractères (OCR). Apparue dès la seconde moitié du XXe siècle, cette technologie dite **déterministe** cherche à isoler des contours pour les comparer à des modèles pré-enregistrés. Ces systèmes requièrent des documents modernes : contraste élevé, polices homogènes et millimétrées. L'OCR classique a été inventé pour lire des formulaires, pas pour déchiffrer l'histoire.

Dès que nous transportons ces outils vers nos sceaux byzantins, le système s'effondre. Un algorithme de détection de contours classique est incapable de distinguer le relief d'une lettre d'une irrégularité du plomb. Le passage du plan au relief constitue une rupture méthodologique majeure. Nous devons abandonner l'idée d'une comparaison simple pour nous tourner vers des architectures capables d'apprendre la variabilité même de l'objet

2. Passage aux méthodes statistiques

Face aux échecs des méthodes géométriques, les années 1990 et 2000 voient émerger les méthodes statistiques (Machine Learning). Nous quitons le monde des règles rigides pour celui des probabilités. Des algorithmes comme les machines à vecteurs de support (SVM), théorisés par

Vladimir Vapnik [19], ont introduit une capacité d'adaptation face aux variations graphiques. [1]

Cependant, cette avancéé masquait une difficulté majeure : La nécessité d'une ingénierie manuelle des caracteritiques (feature engineering). L'humain doit "expliquer" mathematiquement à la machine ce qu'est une lettre, en choisissant manuellement des descripteurs (contours, rapports de forme, etc.). La qualité du résultat dépend entièrement de la pertinence de ces choix. Or, definir manuellement des descripteurs pour des caractères corrodés est une tâche décourageante. Les Modèles de Markov Cachés (HMM), bien qu'efficaces pour les séquences échouent dès que la structure visuelle est trop altérée. Ces méthodes ont revélé les limites d'une description manuelle : l'humain ne peut pas anticiper toutes les variations possibles.

III.L'émergence du Deep Learning en vision par ordinateur

Larrivée de lapprentissage profond (Deep Learning) au début des années 2010 marque un changement de paradigme radical dans le domaine de la vision par ordinateur. Pour la sigillographie et l'étude des documents, cette avancée ne représente pas seulement une amélioration de la précision, mais une véritable mutation dans la manière dont la machine appréhende l'objet historique.

1.Rupture méthodologique

Avec les limites que traîne le *feature engineering* (l'humain doit guider la machine en créant des règles), il fallait des paradigmes nouveaux. La réponse arrive en 2012 lorsqu'un réseau de neurones convolutifs (CNN), développé par Alex Krizhevsky et ses collègues, obtient un taux d'erreur de 15,3 % durant la compétition ImageNet, contre 26,2 % pour le second concurrent. Cet écart n'est pas insignifiant : c'est la chute d'un paradigme.

Là où, pour qu'un ordinateur reconnaisse un chat, on devait lui décrire ce qu'est cet animal et ses caractéristiques pertinentes, aujourd'hui avec les CNN on leur montre naïvement des millions de photos de chat et eux apprennent les représentations directement via ces données brutes, en établissant une grammaire visuelle hiérarchique, des contours simples aux formes complexes.

Pour en revenir à notre cas, dans les approches statistiques classiques mentionnées précédemment (SVM, HMM), le chercheur devait définir lui-même les critères discriminants d'une lettre. Il va s'agir pour lui d'expliquer mathématiquement qu'un Alpha est composé de deux

1. Une machine à vecteurs de support (SVM) est un algorithme de machine learning supervisé qui classe les données en trouvant une ligne optimale ou un hyperplan qui maximise la distance entre chaque classe dans un espace à n dimensions. Toutefois, comme les autres méthodes statistiques de l'époque, elles reposent sur une ingénierie manuelle des descripteurs (*feature engineering*) : l'humain doit choisir quels attributs (contours, rapports de forme, etc.) sont pertinents pour la classification.

segments obliques et d'une barre horizontale. Face à un sceau byzantin où la barre est effacée par l'usure et où les segments sont tordus par la courbure du plomb, ces modèles rigides échouent systématiquement.

Le Deep Learning renverse cette logique. En exposant le modèle à des milliers d'exemples de caractères variés, le réseau de neurones apprend de lui-même à extraire des hiérarchies de caractéristiques :

- Les premières couches du réseau détectent des éléments simples (bords, contrastes, textures du métal).
- Les couches profondes combinent ces informations pour reconnaître des formes abstraites (la silhouette d'un buste de saint ou la structure complexe d'un monogramme).

Comme un enfant qui apprend d'abord à distinguer les lignes avant de reconnaître des visages, le modèle construit lui-même sa propre représentation du monde visuel. Cette capacité d'abstraction est la clé pour traiter le sceau byzantin. Le modèle ne cherche plus une correspondance parfaite avec un modèle idéal, mais il apprend la signature visuelle d'une lettre malgré ses multiples déformations physiques. On passe d'une vision géométrique à une vision contextuelle.

2.Applications au patrimoine culturel

L'entrée de la vision par ordinateur dans le monde des humainités numériques marque un tournant vraiment important. De nos jours on ne se limite plus à stocker des images pour les archiver ; on enseigne,on apprend maintenant aux machines à comprendre et à mettre en valeur ces sources. Pour nos sceaux byzantins, cela signifie passer d'une simple photo inerte à une donnée scientifique vivante et exploitable.

En ayant vu des milliers de caractères, le modèle devient capacable de pratiquer une sorte de reconstruction mentale : Il identifie un "Alpha"ou un "Thêta" même s'il est amputé d'une partie jugée comme illisible.

L'objectif de notre démarche est l'interopérabilité. Pour nous une image de sceau n'a de valeur que si cette image est connectée à son texte et aussi à son contexte. Et donc à travers des modèles comme YOLO, nous pouvons alors isoler chaque caractère et le lier directement à un standard. Notre pipeline ne se limite plus seulement à voir, il transforme une image qui ne parle pas en une fiche d'identité numérique bien structurée.

Il est aussi important de souligner que les algorithmes ne remplacent en aucun cas les experts en sigillographie. Nous travaillons selon le principe d'impliquer l'homme dans le processus que nous appelons "l'humain dans la boucle". On confie les charges lourdes et complexes à la machine : Identifier les zones de texte et proposer une première lecture, une fois que les tâches complexee sont faites. L'historien peut alors se concentrer sur l'essentiel valider les détections

et interpréter le sens historique du document. Cette collaboration garantit des résultats fiables.

IV.Détection d'objets : une reformulation nécessaire pour les sceaux byzantins

Quand on parle de reconnaissance de texte on a tendance à avoir un positionnement de grammairiens, on sous-tend lidée que les caractères soient au préalable isolés dans lobjet cible. Ce prisme, valide pour les documents sur papier, ne lest pas dans le cas des sceaux byzantins. En effet, comme le soulignent les travaux fondateurs de John Nesbitt et Nicolas Oikonomidès dans leur *Catalogue of Byzantine Seals at Dumbarton Oaks and in the Fogg Museum of Art* (19912009) [12], les inscriptions ne suivent pas des lignes continues et délimitées. Les caractères ne forment aucune structure prévisible et présentent localement des niveaux de dégradations avancés. Toute tentative de segmentation textuelle devient donc difficile, se heurtant à labsence de structure prévisible et reconnaissable.

Cette difficulté nous oblige donc à repenser le problème. Plutôt que de soccuper à reconnaître du texte (tâche classique mais pas adéquate dans notre cas), lenjeu consiste donc à localiser puis isoler chaque caractère dans le texte. Dès lors, on fait face à un problème de détection dobjets, au sens technique défini par Zou et al. (2023) [20]. Dans leur synthèse sur lévolution des méthodes de détection dobjets par deep learning, chaque caractère doit être considéré comme un objet unique avec une position dans un espace, une étendue géométrique et une classe sémantique. Ce changement conceptuel est donc fondamental. Il conditionne lensemble des choix architecturaux qui suivront.

1. Les architectures modernes de détection d'objets

Selon Zou, l'évolution des méthodes de détection est marquée par deux époques : avant et après 2012 (voir figure 5). Cette année est associée à la renaissance des approches d'apprentissage profond par des réseaux convolutifs, qui ont graduellement mis fin à la détection d'objets par des algorithmes traditionnels.

Parmi les plus remarquables des approches traditionnelles, on peut notamment citer le *Viola-Jones detector*, originalement développé pour la détection de visages en 2001, mais qui a montré sa pertinence pour d'autres applications. Au lieu de simplement scanner l'image à l'aide d'une fenêtre glissante à plusieurs échelles (opération très coûteuse), les auteurs ont proposé trois techniques innovantes qui ont drastiquement amélioré la performance de l'approche :

— le calcul de l'*intégrale de l'image*, qui permet d'accélérer la convolution en recherchant des valeurs précalculées dans ce que l'on appelle la table des surfaces additionnées ;

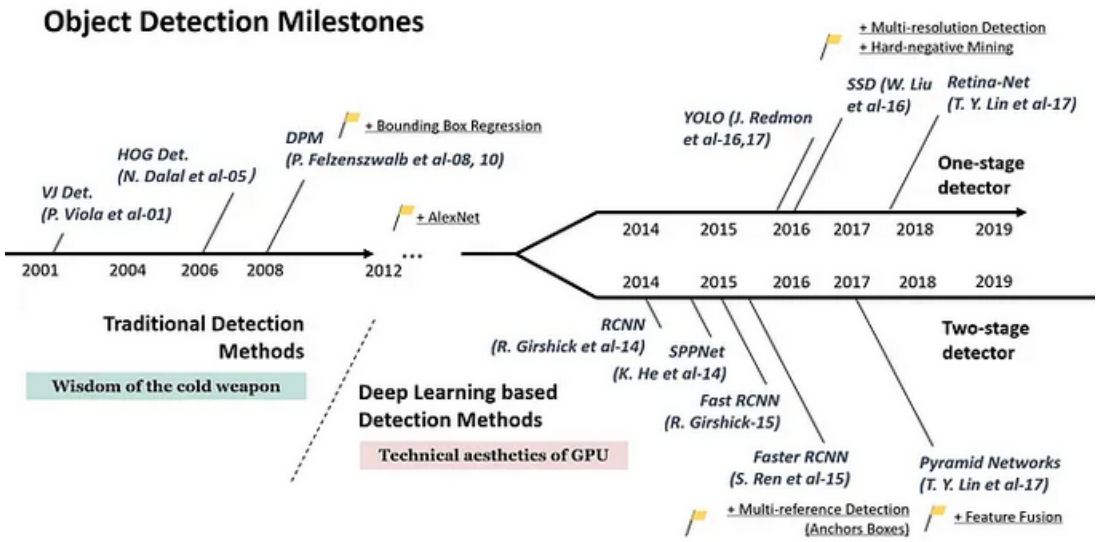


Fig 1. Timeline extrait de l'étude présentée par Zou et al. (2019)

FIGURE 5 – Timeline extrait de l'étude présentée par zou.

— l'utilisation de l'*AdaBoost*, une méthode d'apprentissage itérative qui sélectionne les features les plus pertinents à l'aide d'arbres de décision ;

— la *détection en cascade*, qui est une façon de combiner plusieurs classifieurs complexes en focalisant progressivement sur le premier plan de l'image, plus riche en information.

En 2005, une nouvelle approche, l'*histogramme des gradients orientés* (HOG), a émergé en offrant une solution robuste invariante à l'échelle, et qui est à la base de nombreux algorithmes de détection. Comme son nom l'indique, HOG calcule une carte des histogrammes des gradients de l'image, qui est ensuite pondérée par la moyenne des gradients d'images issues du jeu de données d'entraînement. Ces approches traditionnelles, bien qu'innovantes pour leur époque, ont montré leurs limites face à la variabilité et à la complexité des corpus réels limites que les architectures profondes post-2012 sont venues combler.

Les approches modernes de détections d'objets se divisent en deux grandes familles. La première, celle des détecteurs dits a deux étapes (*two-stage detectors*), introduits par Girshick et al. (2014), [7], puis peaufinés avec Fast R-CNN (2015) et Faster R-CNN (Ren et al., 2015), [16], procède en deux phases. Une qui génère des régions candidates (le modèle parcourt l'image et propose des zones pouvant contenir un objet) et une seconde qui classifie et affine (ici chaque régions candidates est analysée individuellement). Ce procédé offre une précision élevée mais est très coûteux, le rendant inadapté pour les traitements à grande échelle.

La seconde celle des détecteurs dits à une étape (*one-stage detectors*), parmi lesquels on peut retrouver les SSD (*Single Shot MultiBox Detector*),Liu, W. et al. (2016), Lin, T-Y. Et al.(2017) et Redmon, J. et al.(2016), [10], [9] et [14], unifient localisation et classification en une



Fig2. Illustrations de la méthode "Viola-Jones" montrant la sélection des features proéminents

FIGURE 6 – Illustrations de la méthode *Viola-Jones* montrant la sélection des features proéminents.

seule inférence. Cette simplification va permettre un traitement plus rapide, tout en gardant des performances compétitives. Dans le cadre d'un travail avec un corpus potentiellement étendu, le choix de cette famille d'architecture présente d'énormes avantages.

2. La famille YOLO : principe, évolution et pertinence pour notre corpus

YOLO (You only look once) est établi sur un principe fondateur formulé par Joseph Redmon et ses collaborateurs : En une passe du réseau neuronal, effectuer la détection complète. Concrètement le modèle va scinder l'image d'entrée en une grille régulière et va prédire simultanément pour chaque cellule la localisation des objets présents, leurs probabilités d'existence et leur classe d'appartenance. Ainsi, cette conception confère à l'architecture une rapidité d'inférence particulièrement adaptée aux contextes de traitement semi-automatique.

Depuis ses premières versions, la famille YOLO a connu une évolution continue et significative. La version YOLOv3, mise en place par Redmon & Farhadi (2018)[15], introduit la détection à multi-échelle qui repose sur trois niveaux de résolution distincts (grande, intermédiaire, faible), offrant une meilleure prise en charge des objets de petite taille. YOLOv5, paru en 2020 via Ultralytics[8] marque une étape importante en termes d'accessibilité et d'optimisation

pour des environnements de production. YOLOv8 (2023) quant à lui intègre des mécanismes d'attention et une architecture anchor-free[2]+ attention, améliorant encore la précision sur des objets de dimension variable. Cette dernière propriété est particulièrement pertinente pour notre étude, dans la mesure où les sceaux byzantins possèdent des formes et des dimensions très variées selon les lettres et les époques. Une architecture anchor-free comme celle de YOLOv8 est théoriquement la plus adaptée.

Face à des arrières-plans complexes et à des conditions d'acquisition dégradées, des études récentes ont pu étayer la robustesse des architectures de la famille YOLO dans plusieurs contextes d'étude du patrimoine proche du nôtre.

Une étude récente publiée en 2023, appliquée à la détection de monnaies anciennes indonésiennes, à mobilisé YOLOv8 pour localiser, identifier et classifier des pièces présentant des conditions de conservation variés, obtenant une précision de 91%. [17]

Dans le même ordre d'idées, des travaux en numismatique numérique[3], ont révélé que les méthodes de détection automatiques permettent d'analyser des monnaies dégradées du premier millénaire, dont les caractéristiques visuelles se rapprochent de celles des sceaux byzantins.

Ces résultats encourageants confirment la pertinence d'une telle approche pour notre corpus.

3. Travaux appliqués aux sceaux byzantins

Au cours des dernières décennies l'application des réseaux de neurones au documents historiques s'est considérablement développée.

En Reconnaissance de manuscrits médiévaux, le système Transkribus développé par la coopérative européenne (READ-COOP SCE), à représenté une avancée majeure en démocratisant l'accès à des modèles HTR (Handwritten Text Recognition) entraînés sur des corpus historiques variés. Ces travaux ont permis de démontrer qu'une approche mêlant annotations manuelles et apprentissage automatique permettait de se rapprocher de performances significatives, même sur des écrits rares ou peu standardisés.

Dans le cas des inscriptions grecques anciennes, le projet Ithaca paru dans Nature en 2022 et développé par DeepMind en collaboration avec plusieurs universités (Oxford, Venise, Athènes) constitue une référence majeure. Ce modèle, entraîné sur le corpus du Packard Humanities Institute, arrive à restaurer des textes lacunaires, à leur attribuer une provenance géographique

2. Les détecteurs sans ancrage (*anchor-free*) représentent une classe moderne d'architectures de détection d'objets qui identifient et localisent les cibles dans les images sans s'appuyer sur des boîtes de référence prédéfinies. Contrairement aux approches traditionnelles (anchors) qui dépendent d'une grille d'ancrages prédéfinis pour estimer les dimensions, ces modèles prédisent les boîtes englobantes directement à partir des caractéristiques de l'image. Ce changement de paradigme simplifie la conception des modèles, réduit le besoin de réglage manuel des hyperparamètres et aboutit souvent à des architectures plus rapides et plus efficaces, adaptées à l'inférence en temps réel. Les *frameworks* de pointe, notamment *Ultralytics*, ont adopté cette méthodologie pour obtenir une généralisation supérieure sur divers ensembles de données.

et une chronologie avec une grande précision (Assael et al., 2022)[1].

Ithaca est conçu pour aider et élargir le champ d'action des historiens. Si Ithaca atteint à lui seul une précision de 62% dans la restauration de textes endommagés, son utilisation par les historiens a permis d'améliorer leur précision de 25% à 72%, confirmant ainsi l'effet synergique de cet outil de recherche. Bien que se concentrant davantage sur des inscriptions de surface que sur des sceaux, cette recherche illustre le potentiel des modèles d'architectures profondes pour le traitement d'écritures dégradées.

Quoique ces travaux de recherche portent sur des inscriptions gravées en surface, dont la géométrie diffère des sceaux byzantins en relief, ils portent en eux un aveu considérable : les architectures profondes peuvent restaurer des écritures anciennes dégradées, même avec des données lacunaires. Cette capacité à "compléter" l'information manquante, jumelée à la synergie homme-machine observée avec Ithaca, solidifie notre hypothèse de départ : un modèle de détection d'objets comme YOLOv8 pourrait non seulement localiser les caractères sur les sceaux, mais aussi à long terme assister les épigraphistes dans leur interprétation.

En sigillographie numérique, des initiatives comme le projet SigiDoc ont posé les bases d'une infrastructure d'encodage et d'indexation des sceaux byzantins selon les standards TEI. Toutes ces avancées constituent un terreau favorable à l'intégration de méthodes automatiques de reconnaissance, en fournissant des transcriptions de référence exploitables pour l'alignement avec les détections produites par YOLO.

Plus directement encore, les recherches conduites dans le cadre du projet BHAI ont permis d'établir la possibilité d'une segmentation automatique des sceaux en plomb et une première caractérisation des zones dégradées. Eyharabide et al. (2023)[6] ont ainsi posé les jalons d'un traitement computationnel des images sigillographiques, aussi Eyharabide (2024)[5] propose une réflexion plus large sur les défis et perspectives de l'intelligence artificielle appliquée à la sigillographie byzantine. Ces résultats constituent le point de départ direct de notre démarche.

Notre principale difficulté subsiste en l'absence de bases d'images annotées à grande échelle. Un constat partagé par l'ensemble de la communauté intellectuelle travaillant sur des caractères historiques. C'est entre autres cette lacune que vise, à minima, de combler le pipeline proposé dans ce mémoire.

Cet état de l'art met en évidence une convergence progressive entre sciences historiques et modèles d'intelligence artificielle. Les méthodes classiques de reconnaissance ont montré leurs limites face à la complexité que peuvent constituer l'étude de corpus anciens, en raison de leur dépendance à des hypothèses de régularité que les sceaux byzantins ne satisfont pas. Les approches fondées sur l'apprentissage profond, et en particulier les architectures de détection

d'objets de la famille YOLO, offrent en revanche des perspectives adaptées à ce type de corpus.

Toutefois, ces modèles restent tributaires de la disponibilité de données annotées fiables. Dans le cas des sceaux byzantins, cette étape constitue encore le principal verrou scientifique, et c'est sur elle que repose l'essentiel de la contribution méthodologique de ce mémoire.

CHAPITRE II

Méthodologie : Un pipeline semi-automatique

Il ne faut pas perdre de vue que notre pipeline sinscrit dans une démarche de détection et de localisation de caractères présents sur des sceaux byzantins. Autrement dit, l'objectif visé ici est de traiter des tâches de détection dobjets, et produire des bounding boxes autour des caractères identifiés sur l'image cible. Ces tâches constituent le fondement sur lequel reposent les étapes ultérieures de classification et danalyse, que nous aborderons à la suite. Pour réaliser ce travail, nous avons opté pour une architecture qui combine localisation et classification telle que YOLO (You Only Look Once), un modèle de détection en temps réel qui a connu des évolutions considérables depuis sa première version proposée par Redmon et al.(2016)[14].

Nous justifions le choix de ce modèle par le concours de plusieurs facteurs déterminants liés à notre contexte. Premièrement, son architecture single-shot, qui permet un traitement rapide et fluide des images, ce qui constitue un atout non négligeable lorsquon envisage un déploiement à plus grande échelle de notre pipeline sur des corpus de sceaux plus volumineux. Deuxièmement, la variante YOLO de génération 8 variante proposée par Ultralytics en 2023, offre un équilibre assez entre précision de détection et facilité dusage, grâce à une API Python intégrée et une documentation complète.

I. Détection automatique des caractères avec YOLO

1. Constitution du corpus annoté

Le point de départ de tout pipeline de détection repose sur la disponibilité de données annotées de qualité. La qualité, la quantité et la pertinence des annotations d'un *dataset* conditionnent les performances du modèle. En effet, plus les données annotées sont pertinentes, plus les résultats obtenus seront fiables. Dans notre cas, il s'agissait d'annoter manuellement des images de sceaux byzantins afin de fournir au modèle la base de connaissances nécessaire à son apprentissage.

Pour réaliser cette tâche, nous avons retenu la plateforme Supervisely, un outil d'annotation en ligne qui offre une interface ergonomique adaptée aux tâches de vision par ordinateur. Le choix de cette plateforme s'est imposé par sa capacité à gérer des projets collaboratifs, sa compatibilité avec plusieurs formats d'exportation dont le format YOLO, et la possibilité d'annoter à l'aide de différentes formes géométriques (rectangles, polygones, points). Le format dannotation YOLO génère un fichier .txt pour chaque image, contenant des annotations telles que la classe de lobjet, les coordonnées normalisées du centre, la largeur et la hauteur de la boîte englobante (x_center, y_center, width, height), ce qui favorise son adaptation à la détection d'objets. Bien que cette conversion entraîne une perte de précision géométrique par rapport aux polygones d'origine, elle reste adaptée au mode de détection de YOLOv8. Les annotations polygonales originales sont par ailleurs conservées et pourront être exploitées dans de futurs

travaux de segmentation d'instance.

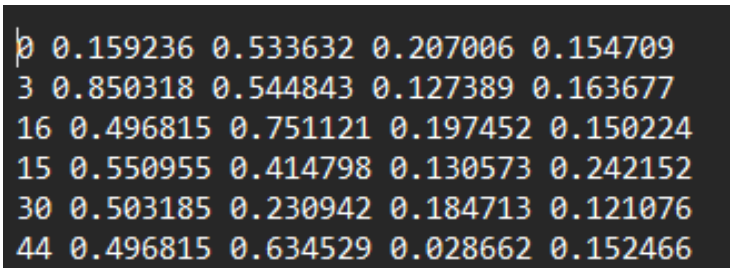


FIGURE 7 – Exemple d'annotations au format YOLO.

Le travail d'annotation, mené conjointement avec l'équipe dannotateurs constituée d'Ézéchiel Sawadogo et Mamadou Barry sous la supervision de l'encadrante et d'un historien, a porté sur un corpus de 351 images de sceaux byzantins. Pour chaque image, les caractères visibles ont été délimités manuellement à l'aide de polygones tracés point par point, épousant aussi fidèlement que possible les contours de chaque lettre. Ce choix d'annotation polygonale, bien que plus coûteux en temps qu'un simple tracé rectangulaire, garantit une représentation plus fidèle de la forme réelle des caractères sur les sceaux. Au total, 49 classes distinctes ont été définies, correspondant aux différents caractères et ligatures identifiés dans le corpus.

L'annotation manuelle s'est avérée un exercice particulièrement exigeant et énergivore, car elle requiert une expertise en sigillographie (même mineure). Les difficultés rencontrées ont nécessité un travail de vérification croisée permanent entre annotateurs afin de garantir la cohérence des annotations produites.

2. Choix de la version et configuration du modèle

Parmi les différentes versions disponibles de l'architecture YOLOv8 (nano, small, medium, large, XLarge), notre choix s'est porté sur la version small. Ce choix est le résultat d'un compromis entre plusieurs contraintes qu'offrait chacune de ces versions. La version small offre un nombre de paramètres suffisant pour capturer la complexité des caractères sur les sceaux (qui, nous le rappelons, présentent des variations typographiques et des superpositions) tout en restant compatible avec les ressources de calcul disponibles, à savoir un GPU T4 sur Google Colab.

Version	Taille	Vitesse	Précision	Usage
YOLOv8n	Nano	Très rapide	Basique	Appareils mobiles
YOLOv8s	**Small**	**Rapide**	**Bonne**	**Notre cas : Colab**
YOLOv8m	Medium	Moyen	Meilleure	Serveurs
YOLOv8l	Large	Lent	Très bonne	GPU puissant
YOLOv8x	XLarge	Très lent	Maximale	Production industrielle

TABLE 1 – Comparaison des variantes de YOLOv8 selon la taille et les performances.

Jusqu'ici nous avons abordé l'architecture YOLOv8 de façon assez générale. Mais qu'est-ce qui rend ce modèle aussi prometteur et intéressant ? Il convient dès lors d'examiner plus en détail les innovations architecturales qui le distinguent des autres modèles. Sa structure anchor-free, déjà citée plus haut, abandonne les boîtes d'ancrage prédéfinies au profit d'une prédiction directe des centres et dimensions des objets. Son *backbone*[3] repose sur une série de blocs C2f (Cross Stage Partial with two convolutions and a flow) qui permettent une extraction de caractéristiques plus riche. Le *neck* utilise une architecture PANet (Path Aggregation Network) bidirectionnelle pour fusionner les informations multiéchelles, tandis que la head de détection opère de manière découplée (decoupled head), séparant la prédiction de classification de celle de localisation.

En ce qui concerne la configuration retenue pour l'entraînement du modèle, les hyperparamètres sont les suivants : une taille d'image d'entrée de 640 x 640 pixels, un *batch size*[4] de 16 (en raison des contraintes mémoire du GPU T4 de colab), un taux d'apprentissage initial de 0.01 avec un scheduler cosine annealing, et un entraînement sur 100 epochs avec early stopping (patience de 50 epochs). Le modèle est initialisé avec des poids pré-entraînés sur COCO (Common Objects in Context), ce qui permet un apprentissage par transfert (transfer learning) particulièrement bénéfique étant donné la taille relativement modeste de notre jeu de données.

3. Stratégie d'entraînement et validation croisée

L'un des principaux enjeux de ce projet réside dans la taille restreinte du corpus d'images annotées. Pour maximiser l'exploitation de nos données et obtenir une estimation robuste des performances du modèle, nous avons mis en place un protocole de validation croisée à 5 plis (5-fold cross-validation).

Ce protocole consiste entre autres à diviser l'ensemble du jeu de données en cinq sous-ensembles (folds) de taille approximativement égale. À chaque itération, quatre folds sont uti-

3. Le terme *backbone* (littéralement colonne vertébrale) désigne la partie principale d'un réseau de neurones chargée d'extraire les caractéristiques visuelles d'une image, des contours simples aux formes complexes.

4. Le terme *batch size* (taille de lot) désigne le nombre d'échantillons de données traités par un système ou un algorithme avant que les paramètres internes ne soient mis à jour.

lisés pour l'entraînement et le cinquième sert de jeu de validation. Autrement dit, on vérifie pendant l'entraînement si les performances du modèle évoluent favorablement. La démarche est répétée cinq fois, de sorte que chaque image serve exactement une fois de donnée de validation. Cette approche présente un double avantage : elle permet d'utiliser lensemble des données pour l'entraînement et la validation, et elle fournit une estimation de la variation des performances, indicateur crucial de la stabilité du modèle. Il convient toutefois de noter qu'en l'absence d'un jeu de test indépendant pour lensemble des données, cette validation croisée fournit à la fois une estimation des performances et une mesure de la capacité de généralisation du modèle.

L'augmentation de données (data augmentation) joue un rôle clé dans notre stratégie d'apprentissage. YOLOv8 dispose nativement dun mécanisme d'augmentation mosaic appliqué par défaut durant lapprentissage. Ce mécanisme rassemble quatre images en une seule mosaïque, contraignant le modèle à détecter des objets dans des contextes variés et à différentes échelles. Entre autres, YOLO applique par défaut des transformations géométriques (translation, mise à l'échelle, retournement horizontal) et des transformations photométriques (teinte, saturation, luminosité) pour simuler la diversité des conditions de reproduction des monogrammes. Ces augmentations sont particulièrement pertinentes dans notre cas, où les reproductions graphiques présentent des orientations variées et des niveaux de contraste hétérogènes.

4. Production automatique de bounding boxes

Après entraînement de notre modèle, portés par la réflexion afin d'évaluer ses capacités de généralisation sur des images inédites, nous avons procédé à une phase d'inférence. Cette méthode permet de produire automatiquement des bounding boxes. Le post-traitement applique un algorithme de Non-Maximum Suppression (NMS) pour éliminer les détections redondantes. Autrement dit, lorsque plusieurs boîtes de détection se chevauchent au-delà d'un seuil d'IoU (Intersection over Union) fixé à 0.5, seule la détection ayant le score de confiance le plus élevé est conservée (on peut faire un parallèle ici avec la méthode naïve où lon choisit l'élément avec la plus grande probabilité). Il a été appliqué dès lors, un seuil de confiance minimal de 0.25 pour filtrer les faux positifs. Nous avons considéré qu'il était préférable d'utiliser un seuil permissif afin de maximiser le nombre de caractères détectés, quitte à accepter quelques faux positifs, plutôt qu'un seuil restrictif qui risquerait d'omettre des caractères réels.

Toutefois, les résultats de notre validation croisée à 5 plis, que nous aborderons dans le chapitre suivant, montrent des performances encourageantes. Malgré les variations sur les différents plis, les scores obtenus témoignent de la capacité du modèle à localiser les caractères de manière fiable sur la majorité des sceaux du corpus.

La détection automatique par YOLOv8s constitue ainsi la pierre angulaire de notre pipeline. Les performances obtenues en validation croisée viendront confirmer la viabilité de cette approche pour le traitement des sceaux byzantins, tout en révélant des axes d'amélioration liés

à l'hétérogénéité du corpus et aux limites propres à la détection par boîtes englobantes. Dans la section suivante, nous aborderons l'étape de classification des caractères détectés, qui vise à identifier le caractère correspondant à chaque région extraite.

II. Étude comparative des architectures de classification

Avec YOLOv8s la détection des caractères est faite par production de régions d'intérêt sous forme de bounding boxes, chacune contenant un caractère. Bien que YOLOv8 assure de façon simultanée la détection et la classification des caractères, nous avons souhaité évaluer si des architectures dédiées à la classification, pourraient aboutir à des performances supérieures sur cette tâche spécifique. Afin de déterminer l'architecture la plus adaptée à cette tâche, nous avons mené une étude comparative entre trois modèles : un CNN entraîné from scratch, ResNet18 par transfer learning, et ViT-Tiny. Cette démarche expérimentale nous a permis de valider empiriquement le choix du meilleur modèle retenu pour le pipeline.

1. Architectures retenues (CNN, ResNet18, ViT-Tiny)

Le choix de ces trois architectures repose sur une progression logique tant en complexité quen paradigme. Pourquoi cela ? En effet, nous avons fini par comprendre que chacune porte une approche distincte de la classification d'images, ce qui permet une comparaison à la fois technique et méthodologique. Le premier challenger retenu est un réseau de neurones convolutif (CNN) conçu et entraîné entièrement from scratch, cest-à-dire sans recours à des poids pré-entraînés. Cette architecture simple sert de ligne de base (*baseline*) pour évaluer les performances minimales quun modèle peut atteindre sur notre jeu de données. Autrement dit, lintérêt est de mesurer ce qu'un réseau relativement peu profond comme un CNN peut apprendre directement à partir de nos données, sans bénéficier de connaissances préalables acquises sur d'autres corpus ; contrairement à ResNet18 ou ViT-Tiny qui bénéficient de connaissances préalables apprises sur ImageNet.

Le deuxième challenger retenu est ResNet18. Il se distingue par l'utilisation de connexions résiduelles (skip connections) qui permettent au gradient de circuler plus efficacement à travers le réseau, résolvant ainsi le problème de la dégradation du gradient (Vanishing gradient) dans les réseaux profonds. Dans notre cas, ResNet18 est utilisé en mode transfer learning. Le modèle est initialisé avec des poids pré-entraînés sur ImageNet, puis sa dernière couche est remplacée (étant donné que le modèle est configuré pour classer les catégories dImageNet) par une couche adaptée à nos 26 classes de caractères. Seules les couches finales sont réentraînées sur notre jeu de données, ce qui permet de tirer pleinement parti des représentations visuelles génériques apprises par le modèle tout en les orientant pour la reconnaissance de caractères.

Le troisième challenger est ViT-Tiny (Vision Transformer), qui sappuie sur une adaptation des architectures Transformer, initialement conçues pour le traitement du langage naturel, au domaine de la vision par ordinateur. Introduit par Dosovitskiy et al. (2021), le Vision Transformer découpe l'image d'entrée en petits carrés de taille fixe quon appelle *patches*, les projette en séquences de vecteurs, puis applique un mécanisme d'auto-attention (self-attention) pour capturer les relations entre les différents carrés de l'image. Par comparaison, chaque patch va regarder les autres patches et identifier les liens avec les différentes régions de limage. ViT-Tiny est la variante la plus légère, elle aussi utilisée en transfer learning. L'intérêt de cette architecture est d'évaluer si le mécanisme d'attention, capable de capturer des dépendances à longue portée, offre un avantage sur les approches convolutives (CNN) pour la reconnaissance de caractères présentant des variations morphologiques importantes.

2. Prétraitement et construction du jeu de données

Le jeu de données utilisé pour la classification a été constitué à partir des annotations manuelles que nous avons réalisées sur Supervisely. À l'aide d'un script Python, chaque caractère annoté a été extrait de l'image originale du sceau sous forme d'imagette individuelle, puis rangé dans un dossier correspondant à sa classe. Au total, environ 2 196 imagettes ont été extraites et réparties en 26 classes distinctes.

On notera que le nombre de classes diffère entre la phase de détection (49 classes) et la phase de classification (26 classes). Cette réduction s'explique par un travail de nettoyage du jeu de données. Les classes contenant un nombre insuffisant d'échantillons ont été écartées afin de garantir un volume d'entraînement minimal par classe. Nous navons gardé que les classes avec un nombre suffisant d'échantillons pour permettre un apprentissage fiable. La méthode de nettoyage n'a pas été appliquée à la détection, où l'ensemble des 49 classes a été conservé. En effet, les deux tâches ne poursuivent pas le même objectif. La localisation est moins sensible au déséquilibre des classes que la classification, qui demande un nombre conséquent d'exemples par classe pour distinguer les morphologies proches.

Un prétraitement a été appliqué aux imagettes avec un redimensionnement à une taille uniforme correspondant à nos architectures (224 X 224 pixels pour ResNet18 et ViT-Tiny, 64 X 64 pixels pour le CNN). Le jeu de données a été ensuite divisé en ensembles d'entraînement et de test selon le ratio 80/20, avec une stratification par classe pour préserver la distribution des caractères dans chaque ensemble.

3. Protocole d'évaluation

L'évaluation des trois architectures sest faite avec un protocole classique permettant une comparaison équitable. Chaque modèle est entraîné sur le même ensemble d'entraînement et

évalué sur le même ensemble de test, avec les mêmes conditions de prétraitement des données.

Les métriques retenues pour l'évaluation sont l'accuracy, le F1-score et la matrice de confusion, qui permet d'identifier les classes les plus problématiques et les confusions récurrentes entre caractères à morphologie proche.

Les résultats détaillés de cette étude comparative, incluant les performances de chaque modèle et l'analyse des confusions, seront présentés dans le chapitre consacré à la discussion. Nous pouvons néanmoins indiquer que cette démarche comparative a permis de valider le choix de l'architecture retenue pour le *pipeline*, en s'appuyant sur des critères de performance, de stabilité et de temps dentraînement.

CHAPITRE III

Expérimentations et résultats

Ce chapitre présente les résultats obtenus lors de l'évaluation du module de détection des caractères de notre *pipeline*, qui s'appuie sur YOLOv8s. Les conditions de collecte des données, les conditions expérimentales, les métriques et le protocole de validation ayant été détaillés au chapitre précédent, nous nous concentrons ici sur l'analyse des performances obtenues.

I. Résultats quantitatifs de la détection

1. Résultats par fold

Le modèle YOLOv8s a été entraîné et évalué selon un protocole de validation croisée à 5 plis (5-fold cross-validation), avec une répartition aléatoire des images entre les ensembles d'entraînement et de test afin de sassurer que la même image n'apparaisse dans le test de plusieurs fold, évitant ainsi toute fuite de données entre les ensembles dentraînement et de test. Chaque pli a été entraîné pendant 100 epochs sur Google Colab (GPU T4), selon les hyperparamètres détaillés au chapitre précédent. Le tableau 2 ci-dessous présente les résultats obtenus pour chaque fold. Ce protocole permet également de détecter d'éventuels phénomènes de surapprentissage en comparant les performances entre les différents plis. 2

Fold	**mAP50**	**mAP50-95**	**Precision**	**Recall**
Fold 0	0.7876	0.69	0.78	0.73
Fold 1	0.7406	0.65	0.76	0.70
Fold 2	0.7219	0.63	0.74	0.69
Fold 3	0.7826	0.68	0.79	0.74
Fold 4	**0.8554**	**0.75**	**0.87**	**0.83**
Moyenne	**0.7776**	**0.68**	**0.78**	**0.73**
Écart-type	$\pm$0.046	$\pm$0.04	$\pm$0.04	$\pm$0.05

TABLE 2 – Résultats de la détection YOLOv8s par fold (validation croisée 5-fold)

La mAP@50 moyenne sur l'ensemble du jeu de données s'établit à 0,778 avec un écart-type de 0,046, ce qui témoigne dune stabilité raisonnable du modèle malgré la taille réduite du corpus (351 images). Le fold 4 se démarque clairement avec une mAP@50 de 0,856, suggérant que son sous-ensemble de test contenait des images de sceaux assez claires avec des classes mieux représentées. À l'inverse, les folds 1 et 2, affichant des scores inférieurs à 0,75, reflètent vraisemblablement la présence de caractères rares dans leur sous-ensemble de test.

La précision moyenne (0,78) et le rappel moyen (0,73) révèlent un compromis caractéristique : le modèle tend à être légèrement plus conservateur dans ses détections, ne conservant que les caractères dont il est sûr (le seuil de confiance, voir le chapitre précédent) au détriment

d'une couverture exhaustive. Ce comportement est cohérent avec un corpus composé de reproductions graphiques de sceaux où les caractères superposés et entrelacés rendent la détection difficile.

2. Courbes et visualisations

Afin de rendre compte du comportement du modèle au cours de l'apprentissage, nous proposons ici une analyse quantitative basée sur les courbes d'entraînement et des visualisations complémentaires. Leur analyse et leur interprétation vont nous permettre de vérifier la stabilité du modèle et de détecter d'éventuels phénomènes de surapprentissage.

2.1 Courbes d'entraînement

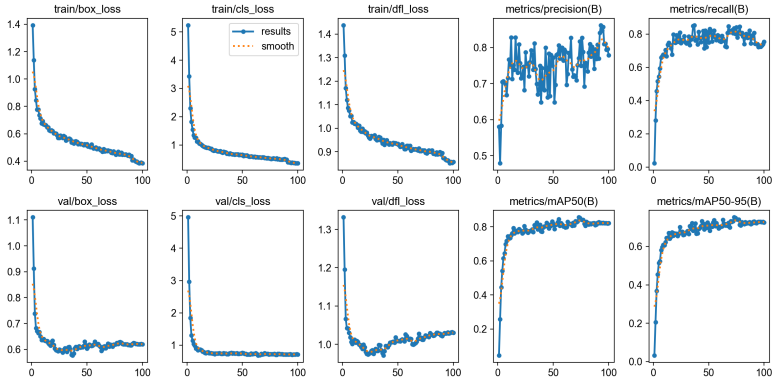


FIGURE 8 – Courbes d'entraînement et métriques de validation, fold 4 (100 époques)

Les courbes dentraînement générées par YOLOv8 fournissent une vision complète de la dynamique dapprentissage. La figure 8 présente lévolution des pertes (box loss, classification loss, DFL loss) ainsi que les métriques de performance (précision, rappel, mAP@50, mAP@50-95) au fil des 100 epochs pour le fold 4.

Les trois fonctions de perte diminuent rapidement au cours des 20 premières epochs, puis se stabilisent. Les pertes de validation suivent la même tendance, elles décroissent de manière régulière sans remonter, ce qui confirme que le modèle ne souffre pas de surapprentissage. La La mAP@50 atteint un plateau autour de 0,85 dès l'epoch 30, tandis que la mAP@50-95, plus exigeante, converge vers 0.75.

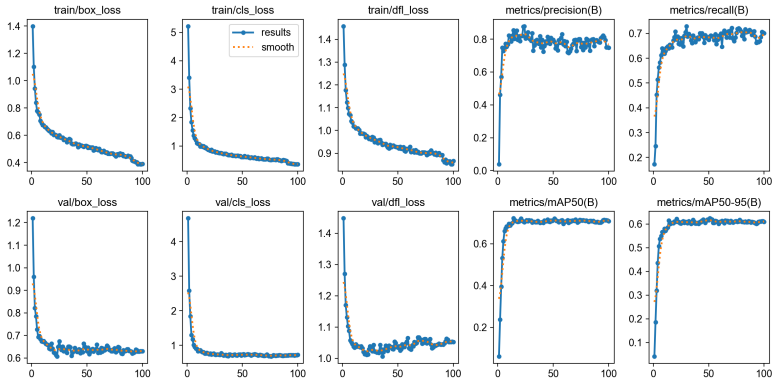


FIGURE 9 – Courbes d'entraînement et métriques de validation, fold 2 (100 époques)

Par opposition au fold précédent les trois fonctions de perte d'entraînement diminuent rapidement au cours des 20 premières epochs, suivant un profil similaire au fold 4. Les pertes de validation décroissent également mais présentent davantage d'oscillations, notamment la `val/box_loss` qui fluctue entre 0.62 et 0.68 sans jamais se stabiliser complètement. Ce comportement suggère que le modèle peine à converger de manière stable sur ce sous-ensemble de données.

La précision montre des fluctuations importantes tout au long de l'entraînement, oscillant entre 0.65 et 0.85, ce qui contraste avec le fold 4 où la précision se stabilise autour de 0.87. Le recall suit une tendance ascendante mais plafonne autour de 0.70, nettement en dessous du 0.83 atteint par le fold 4.

La mAP@50 atteint un plateau aux alentours de 0.72 dès l'epoch 30, soit environ 13 points en dessous du fold 4 (0.85). La mAP50-95 converge vers 0.63, confirmant des performances globalement inférieures.

2.2 Courbes Precision-Recall

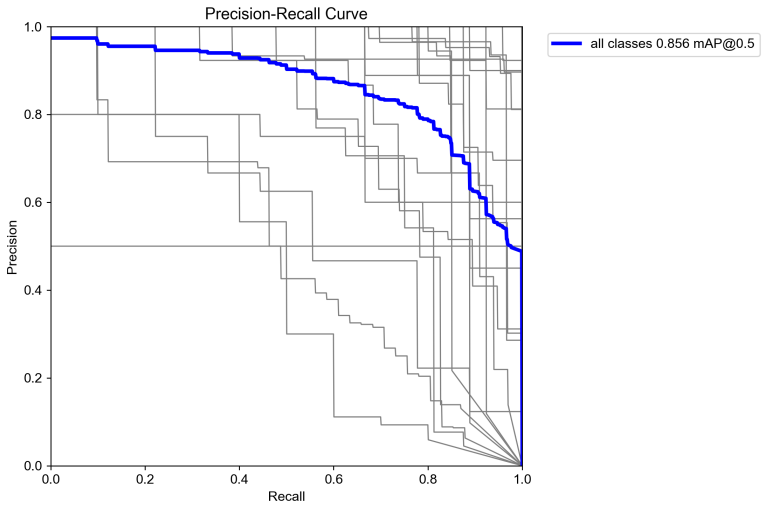


FIGURE 10 – Courbe Precision-Recall, fold 4 (mAP@0,5 = 0,856)

La courbe Precision-Recall synthétise le compromis entre la capacité du modèle à ne détecter que des vrais positifs (Precision) et sa capacité à retrouver tous les caractères présents (Recall). L'aire sous la courbe bleue correspond à la mAP50 = 0.856. La courbe reste élevée jusqu'à un Recall d'environ 0.6, puis décroît progressivement, ce qui est typique d'un détecteur confronté à des classes rares ou visuellement ambiguës. Les courbes grises individuelles montrent une forte variabilité inter-classes.

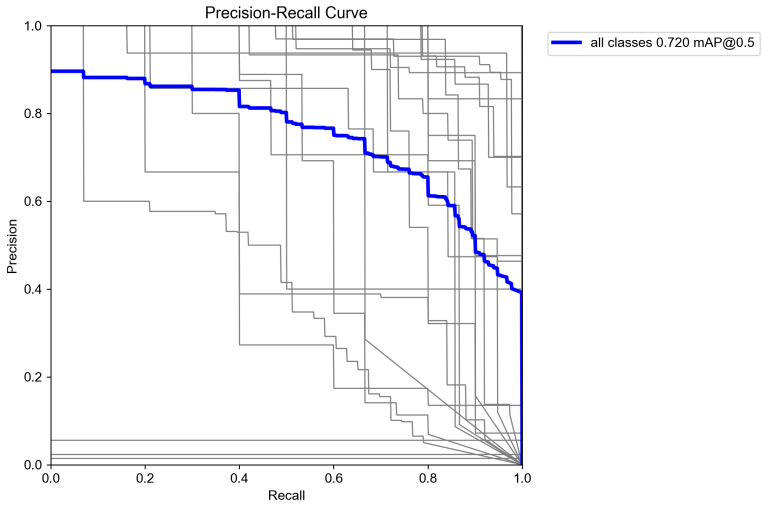


FIGURE 11 – Courbe Precision-Recall, fold 2 (mAP@0,5 = 0,720)

La courbe Precision-Recall du fold 2 confirme les performances inférieures observées sur ce pli. La courbe bleue globale décroît de manière plus prononcée que celle du fold 4. Dès un recall

de 0.4, la précision commence à chuter, passant sous 0.80, alors que le fold 4 maintenait une précision élevée jusqu'à un recall de 0.6. Cette chute plus précoce indique que le modèle doit accepter davantage de faux positifs pour retrouver les caractères présents dans ce sous-ensemble de test.

Les courbes grises individuelles révèlent une variabilité inter-classes encore plus marquée que sur le fold 4. Plusieurs courbes grises restent proches de 0 sur l'ensemble du graphique, ce qui signifie que certaines classes ne sont quasiment jamais détectées correctement.

À l'inverse, quelques classes fréquentes maintiennent des courbes hautes proches de 1, confirmant que le modèle reste performant sur les caractères bien représentés dans le corpus. Cet écart entre classes illustre l'impact du déséquilibre du corpus sur les performances, et renforce le constat que l'enrichissement des classes rares constitue un axe d'amélioration prioritaire.

2.3 Courbes de confiance F1

La courbe F1-Confidence représente le score F1 en fonction du seuil de confiance. Elle permet d'identifier le seuil à partir duquel le modèle offre le meilleur équilibre entre précision et rappel.

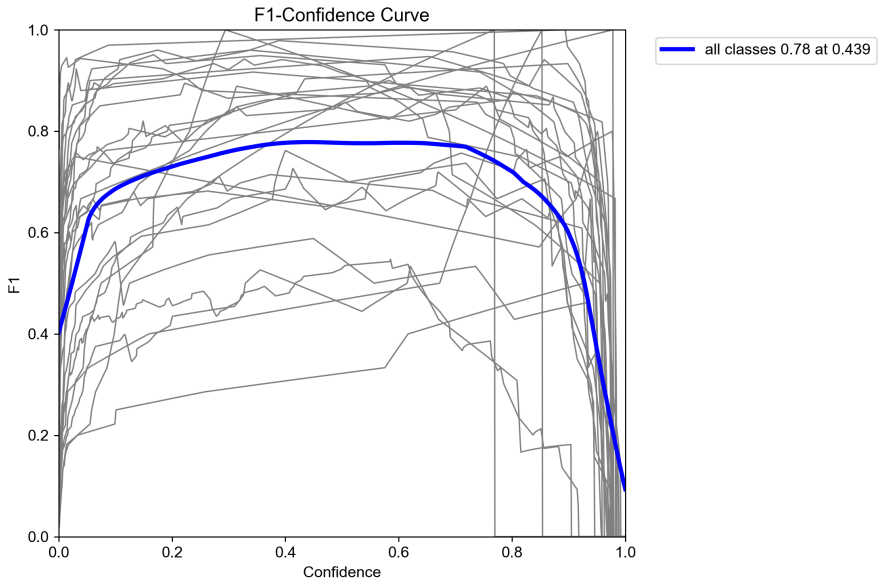


FIGURE 12 – Courbe F1-Confidence, fold 4 (F1 = 0,78 au seuil optimal de 0,439)

La courbe F1-Confidence indique le score F1 (moyenne harmonique de la précision et du rappel) en fonction du seuil de confiance appliqué aux détections. Le F1 maximal de 0.78 est atteint au seuil de 0.439, ce qui signifie que le modèle offre le meilleur équilibre entre précision et rappel lorsqu'il ne conserve que les détections dont la confiance dépasse 43.9%.

La courbe F1-Confidence du fold 2 indique un F1 maximal de 0.67, atteint au seuil de

confiance de 0.327. Le modèle offre donc son meilleur équilibre entre précision et rappel lorsqu'il ne conserve que les détections dont la confiance dépasse 32.7%. Ces valeurs sont inférieures à celles du fold 4 (F1 = 0.78 au seuil de 0.439), ce qui signifie que le modèle est globalement moins confiant dans ses détections sur ce sous-ensemble de test. Les courbes grises individuelles montrent une dispersion importante entre les classes, confirmant que les performances du fold 2 sont pénalisées par la présence de classes plus difficiles à détecter.

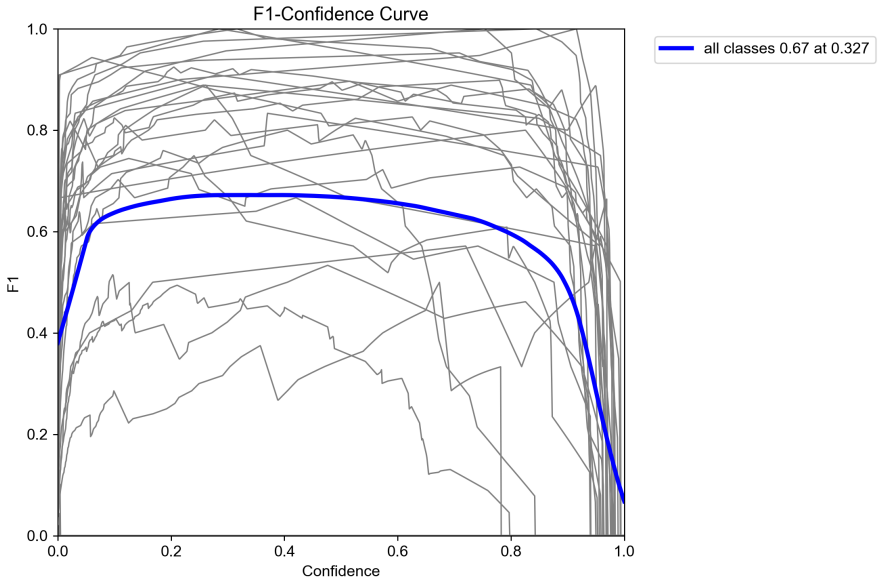


FIGURE 13 – Courbe F1-Confidence, fold 2 (F1 = 0,67 au seuil optimal de 0,327)

3. Matrice de confusion et analyse par classe

La matrice de confusion normalisée (figure 14) constitue lun des outils d'analyse les plus informatifs pour un problème de détection multi-classes. Avec 49 classes de caractères, elle nous permet de mettre en lumière les confusions systématiques entre caractères visuellement proches.

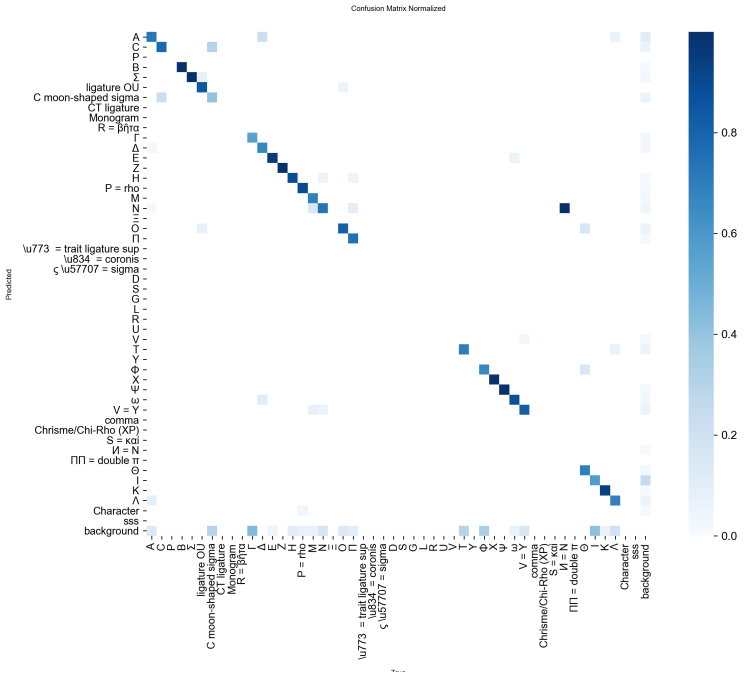


FIGURE 14 – Matrice de confusion normalisée, fold 4

Plusieurs observations se dégagent de cette matrice. Les caractères fréquents et morphologiquement distincts, tels que A, E, O, P=rho et V=Y, affichent des taux de classification correcte élevés sur la diagonale.

En revanche, des confusions récurrentes apparaissent entre certaines paires de caractères à savoir le B qui est parfois confondu avec Σ, ce qui sexplique par la petite ressemblance morphologique de ces deux lettres sur des monogrammes. De même, le N est régulièrement associé à la classe background, indiquant que ses barres verticales fines sont parfois ignorées par le modèle.

La dernière ligne de la matrice, correspondant à la classe *background*, montre les caractères que YOLO a complètement manqués. Ces faux négatifs se concentrent principalement sur les classes rares et les caractères de petite taille.

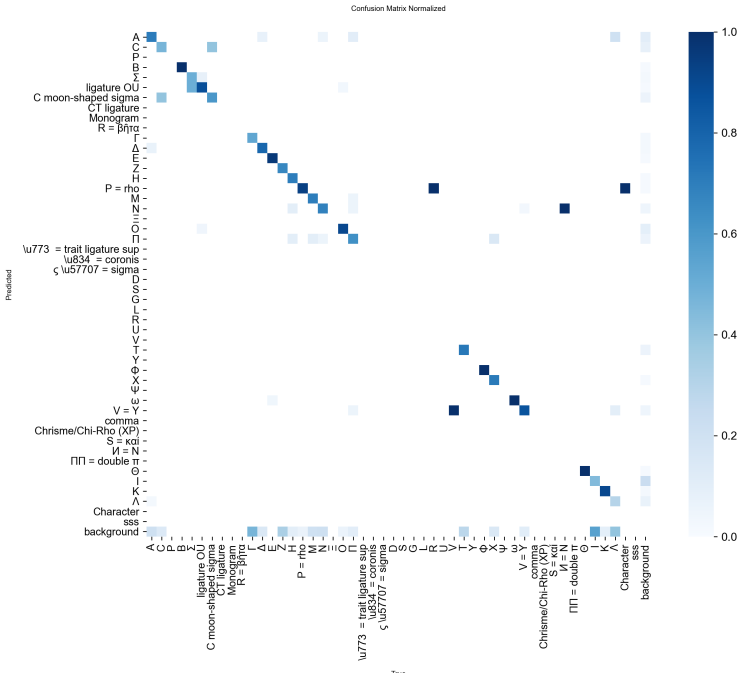


FIGURE 15 – Matrice de confusion normalisée, fold 2

La matrice de confusion normalisée du fold 2 révèle des tendances similaires au fold 4, mais avec des différences notables. Sur la diagonale, les caractères fréquents tels que A, E, ligature OU et C moon-shaped sigma maintiennent des taux de classification correcte élevés. Cependant, la diagonale apparaît globalement moins marquée que celle du fold 4, ce qui traduit un taux d'erreur plus important sur l'ensemble des classes.

On observe que la confusion B/Σ persiste, confirmant qu'il s'agit d'un problème structurel lié à la ressemblance morphologique de ces deux caractères et non d'un artefact propre à un seul fold. Le P = rho est confondu avec la classe V = Y et la classe background, indiquant que ce caractère est tantôt mal classifié, tantôt complètement ignoré par le modèle.

La dernière ligne de la matrice, correspondant à la classe background, est nettement plus active que sur le fold 4 : davantage de caractères réels sont classés comme fond. Cette observation est cohérente avec le recall plus faible du fold 2 (0.69 contre 0.83 pour le fold 4) et confirme que le modèle manque un nombre plus important de caractères sur ce sous-ensemble de test.

En synthèse, l'analyse croisée des matrices de confusion des folds 2 et 4 met en évidence deux types de difficultés. D'une part, des confusions structurelles entre caractères morphologiquement proches (B/Σ, P = rho/V = Y), qui se retrouvent indépendamment du fold et nécessiteraient un enrichissement ciblé du corpus pour ces classes. D'autre part, une sensibilité à la composition du sous-ensemble de test, les folds contenant davantage de classes rares ou de monogrammes complexes affichant des performances significativement inférieures. Ces observations confirment que l'hétérogénéité du corpus et le déséquilibre des classes constituent les principaux axes d'amélioration pour de futurs travaux.

II. Résultats qualitatifs de la détection

Dans cette section, nous présentons une analyse qualitative des résultats obtenus lors de la validation croisée, en nous concentrant spécifiquement sur les deux folds nous servant d'exemples, à savoir le fold 4, qui correspond au meilleur score de performance, et le fold 2, qui affiche les performances les plus faibles. Plutôt que de s'étaler sur l'ensemble des résultats, ce choix permet de mettre en lumière de manière contrastée le comportement du modèle dans des conditions optimales et défavorables. Cette approche comme nous l'avons vu facilite l'interprétation des facteurs influençant la détection, en identifiant les forces et les limites du modèle selon la composition des données dévaluation.

L'examen visuel des prédictions permet de compléter l'analyse quantitative. Les figures ci-dessous présentent, pour le fold 4 (meilleur fold), une comparaison entre les annotations de référence (ground truth) et les détections automatiques produites par le modèle. Chaque bounding box est accompagnée du nom de la classe prédite et dun score de confiance.

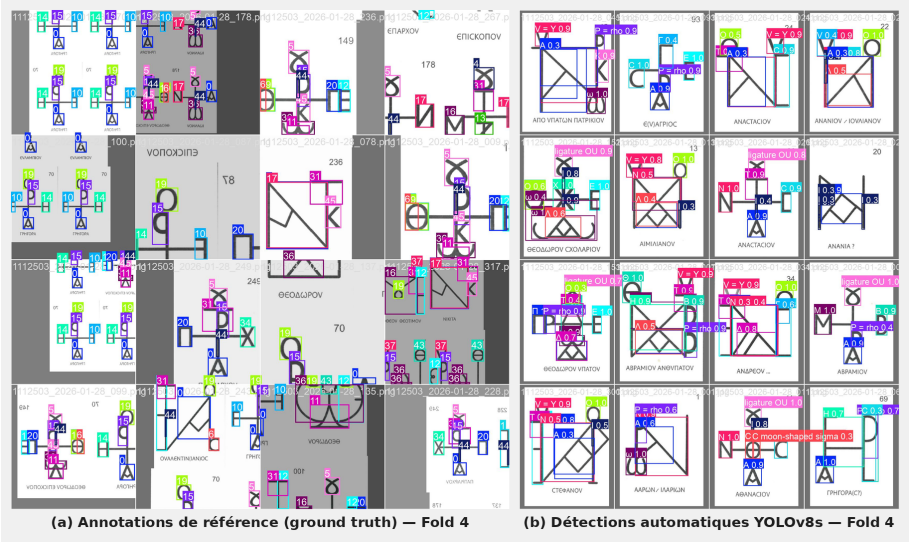


FIGURE 16 – Comparaison entre les annotations de référence et les détections automatiques YOLOv8s, Fold 4

On observe que le modèle parvient à une détection correcte de la majorité des caractères bien individualisés, avec des scores de confiance souvent supérieurs à 0,8 pour les lettres fréquentes telles que A, O, V=Y ou la ligature OU. Il apparaît que les caractères de grande taille et bien contrastés bénéficient de détections précises, tant en localisation qu'en classification.

En revanche, certaines difficultés récurrentes apparaissent. Les barres verticales fines de type Iota (I), la lettre A quand elle est implicite dans le monogramme sont détectées avec des scores de confiance plus faibles (entre 0,3 et 0,6), reflétant l'ambiguïté de ces éléments qui peuvent être confondus avec des traits structurels du monogramme ou dautres caractères.

Par ailleurs, les sceaux présentant une forte agglutination des caractères montrent des détections manquantes, confirmant l'impact de la complexité structurelle des monogrammes sur les performances du modèle.

À l'inverse, le fold 2 affiche la mAP50 la plus faible (0.7219), avec une précision de 0.74 et un recall de 0.69. La comparaison entre les annotations de référence et les prédictions du modèle sur ce fold met en évidence plusieurs difficultés. On constate d'abord un nombre significatif de détections manquantes. Des caractères clairement visibles dans les annotations de référence ne sont pas détectés par le modèle. Les scores de confiance sont globalement plus faibles, se situant fréquemment entre 0.3 et 0.5, là où le fold 4 produisait des scores supérieurs à 0.8 pour des caractères similaires.

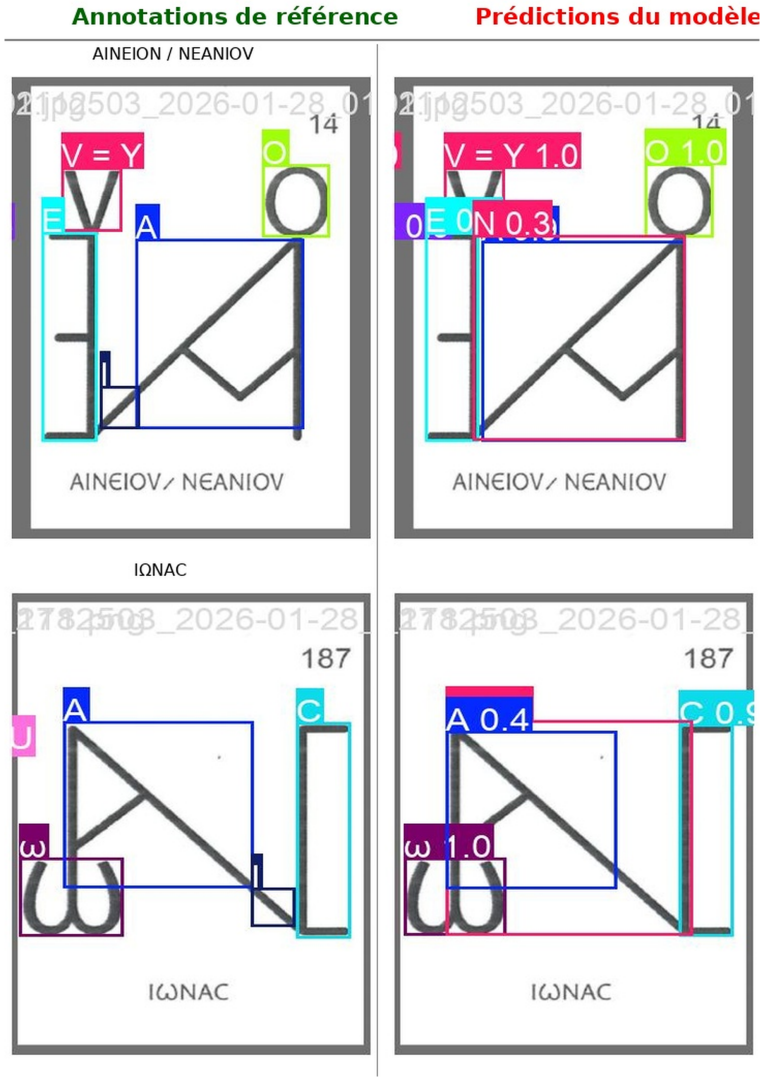


FIGURE 17 – Comparaison entre les annotations de référence et les détections automatiques YOLOv8s, Fold 2

Les barres verticales fines de type Iota (I) restent problématiques sur les deux folds, mais

de manière plus marquée sur le fold 2, qui l'ignore complètement. Cette disparité entre les deux folds s'explique vraisemblablement par la composition de leurs sous-ensembles de test respectifs.

Au regard de l'ensemble des observations formulées sur les folds 2 et 4, il convient de replacer ces résultats dans le contexte plus large de la distribution des données au sein du corpus. La figure 18 apporte un éclairage complémentaire sur les facteurs structurels qui conditionnent les performances du modèle.

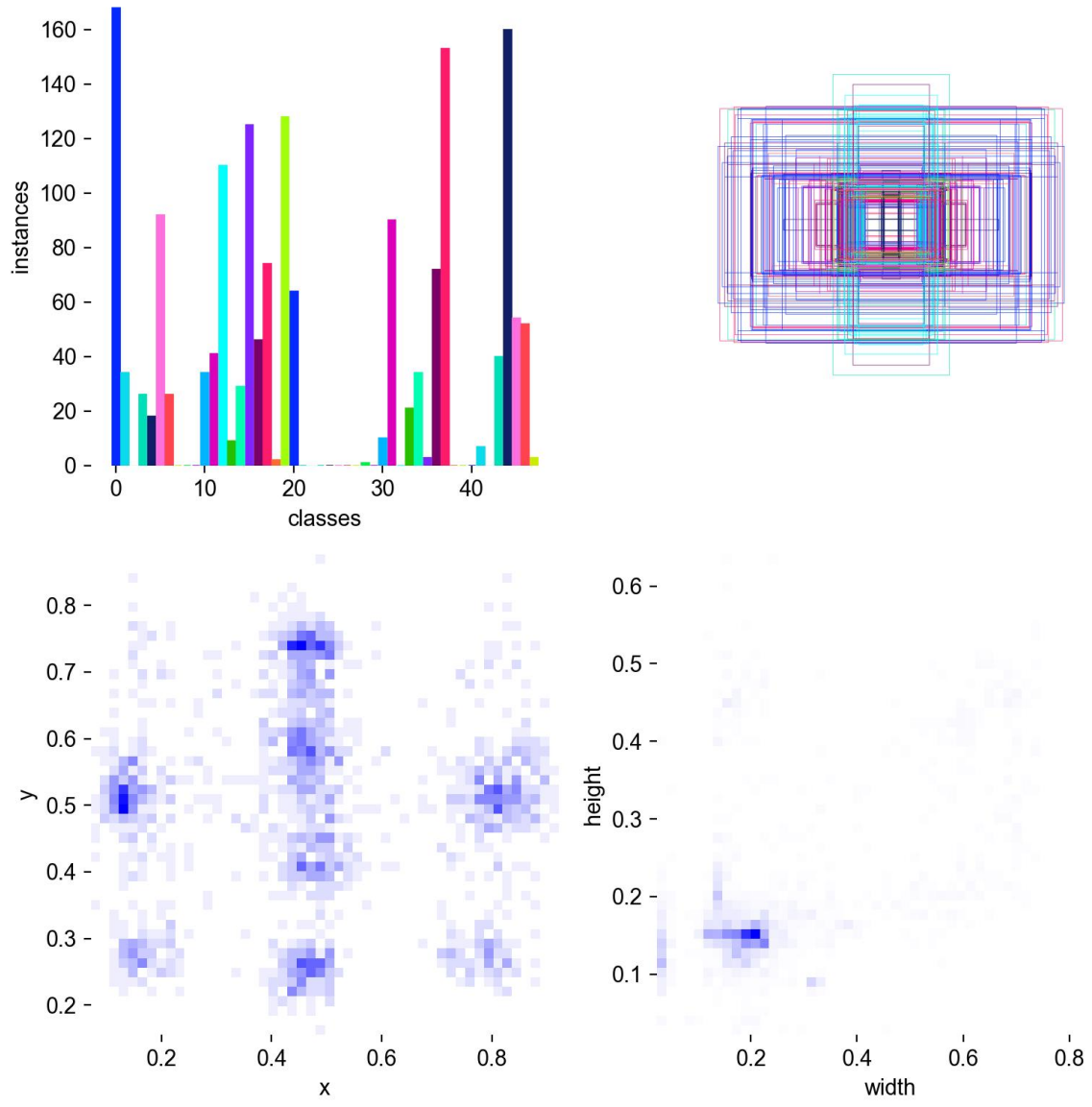


FIGURE 18 – Distribution des classes, fold4

La figure confirme le déséquilibre important entre les classes on remarquera que quelques caractères fréquents dépassent 150 occurrences tandis que de nombreuses classes ne comptent

que quelques instances. Ce déséquilibre, affecte directement la capacité du modèle à généraliser sur les classes rares. La carte de densité des positions révèle trois zones de concentration correspondant aux emplacements typiques des inscriptions sur les sceaux byzantins (partie supérieure, centrale et inférieure), tandis que les bounding boxes se concentrent autour de dimensions relativement uniformes (largeur et hauteur réduites), reflétant la taille caractéristique des lettres gravées.

On notera que ces observations mettent en lumière les facteurs structurels qui conditionnent les performances du modèle. Les limites identifiées au fil de cette analyse seront examinées de manière plus approfondie dans la section suivante.

III. Limites observées et analyse des erreurs

L'analyse de nos résultats, au delà des mesures globales de performance, met en lumière les défis réels auxquels se confronte l'automatisation de supports aussi complexes que les sceaux byzantins. En examinant les échecs du modèle, nous avons identifié trois types de limites qui ralentissent encore la précision absolue du *pipeline*.

Le poids du déséquilibre des classes : La première limite, sans doute la plus marquante, est d'ordre statistique. Notre corpus de 351 images est sujet à ce qu'on pourrait appeler la "longue traîne" : une poignée de caractères (A,E,O,) saturent l'espace, alors que d'autres ne sont représentés que par de rares occurrences($\Xi, \Psi, \zeta, \Phi, \Omega, X$). Cette hétérogénéité explique la force de la variation entre nos plis de validation. Par exemple, le fold4 grimpe à une mAP@50 de 0,856 car il bénéficiait d'une répartition plus favorable, tandis que le fold2 stagne à 0,721. Pour les caractères rares, YOLOv8s manque de point de comparaison pour construire une signature visuelle stable. On observe ainsi que pour environ 15% des classes, le modèle échoue presque systémetiquement, les confondant avec le bruit de fond (background) ou des caractères voisins.

Ambiguïtés morphologiques et complexité des monogrammes : Le deuxième niveau de difficulté est lié à la nature structurelle même du monogramme. La matrice de confusion montre des confusions récurrentes entre des caractères partageant des formes presque identiques. Le B(beta) par exemple, est souvent, confondu avec le Σ(sigma) ou même avec les R, surtout lorsque le monogramme présente une partie d'un caractère qui n'apparaît presque pas. Un point particulièrement frustrant concerne les caractères N(Nu) et I(iota) : les barres verticales très fines de ceux-ci sont souvent ignorées par le modèle, qui les classe en "background". Le réseau de neurones semble interpréter ces traits légers comme de simples rayures ou des irrégularités du monogramme plutôt comme des éléments d'écriture. De plus, les monogrammes cruciformes posent un problème topologique : les lettres s'y chevauchent tellement qu'il devient difficile pour l'algorithme de délimiter des boîtes de détection (bounding boxes) distinctes. Le modèle

voit la forme globale, mais peine à isoler chaque caractère qui la compose.

Limites propres à la détection par boîtes englobantes : Enfin, nous trouvons que la limite majeure réside dans le paradigme même de la détection par boîtes englobantes utilisé par YOLOv8s. L'algorithme cherche à isoler chaque caractère dans un rectangle défini par des coordonnées précises. Or, les monogrammes(notamment cruciformes) posent un problème topologique majeur : les lettres ne sont pas alignées, mais s'entremêlent et se superposent sur des axes communs.

Cette architecture crée une ambiguité pour l'algorithme : lorsque deux ou plusieurs lettres partagent une même région verticale, leurs boîtes englobantes se chevauchent presque totalement. Le modèle, dont les filtres de convolution sont conçus pour distinguer des objets séparés, peine alors à isoler chaque caractère qui compose la forme globale. Il en résulte soit une omission(le modèle ne prédit qu'une seule boîte pour deux lettres fusionnées), soit une imprécision de localisation où la boîte englobe des pixels appartenant à plusieurs caractères distincts. En l'absence de contours nets et isolés, la segmentation rectangulaire imposée par les bounding boxes atteint ses limites techniques face à la densité visuelle du monogramme.



FIGURE 19 – Exemple de deux ou plusieurs lettres qui partagent une même région verticale.

IV. Synthèse des résultats

Malgré les limites évoquées, cette phase d'expérimentation souligne la viabilité de notre approche. Avec une mAP@50 moyenne de 0,78, le module de détection prouve qu'il peut devenir un assistant précieux pour le déchiffrement épigraphique.

Un modèle "prudent" mais fiable : L'un des enseignements majeurs est le compromis que le modèle a établi entre la précision (0,78) et le rappel (0,73). YOLOv8s a développé un

comportement plutôt conservateur : il préfère ne pas détecter un caractère s'il a un doute, plutôt que de proposer une identification erronée. Pour un chercheur, c'est un avantage considérable. Il est en effet beaucoup plus rapide de compléter manuellement les quelques lettres manquantes (omissions) que de devoir repasser derrière la machine pour corriger des erreurs de lecture (faux positifs) qui pourraient fausser toute l'interprétation du sceau.

Convergence et robustesse technique : L'analyse des courbes d'apprentissage nous rassure sur la stabilité technique du projet. La convergence rapide dès la 30ème epoch montre que la stratégie de transfert d'apprentissage (Transfer Learning) a parfaitement fonctionné. Même avec un dataset restreint, le modèle a réussi à se spécialiser sur l'alphabet byzantin sans tomber dans le piège du sur-apprentissage (*overfitting*). Les fluctuations observées sur certains plis de validation ne remettent pas en cause l'architecture, mais soulignent simplement que la performance reste, pour l'instant, très dépendante de la qualité iconographique des échantillons de test.

En conclusion, ces résultats fixent une base solide pour la suite de notre travail. La courbe F1-Confidence nous permet de définir un seuil d'intervention idéal à 0,439, garantissant le meilleur équilibre pour les tests en conditions réelles. Pour aller plus loin, il sera crucial d'explorer des pistes comme l'augmentation de données synthétiques pour "nourrir" les classes rares. En l'état, le module de détection est opérationnel et prêt à alimenter les étapes de reconstruction textuelle.

CHAPITRE IV

Discussion

I. Apports du pipeline proposé

l'intérêt majeur du *pipeline* semi-automatique que nous avons conçu réside dans sa capacité à faire le pont entre la rigueur de la sigillographie traditionnelle et la puissance de calcul des modèles neuronaux modernes. Loin d'être une simple application technique, notre démarche apporte des solutions concrètes à des verrous, des blocages scientifiques identifiés au cours de nos recherches, notamment le manque de données annotées et la difficulté d'accès à certaines sources (SACADO).

Notre travail se veut être une réponse directe à la raréfaction de l'expertise sigillographique face au volume colossal des sources. Nous avons constaté systématiquement que les sceaux restent tributaires d'un travail manuel et chronophage qui ne permet pas, à lui seul, d'absorber les dizaines de milliers de pièces restant à étudier. En automatisant la localisation des caractères grecs, notre *pipeline* devient un véritable assistant cognitif. En optant pour l'architecture YOLOv8s, nous proposons un modèle capable de traiter les images avec une rapidité d'inférence indispensable pour envisager, à long terme, un déploiement sur des corpus beaucoup plus vastes.

Sur le plan technique, nous avons dû confronter notre modèle à la réalité d'un objet que nous qualifions de **rebelle** : le sceau en plomb. Contrairement aux systèmes d'OCR classiques qui s'effondrent face au relief et aux supports dégradés, notre approche par apprentissage profond permet à la machine d'apprendre la **signature visuelle** d'un caractère malgré la corrosion, l'usure ou les déformations géométriques dues à la sphéricité du flan métallique. L'un de nos choix méthodologiques forts a éte de configurer un modèle assez prudent en privilégiant la precision au rappel. Nous avons ainsi, obtenu un système fiable qui minimise les faux positifs. Pour un chercheur, il est en effet bien plus efficace de compléter manuellement quelques omissions que de devoir corriger des erreurs de lecture systématiques qui fausseraient l'interprétation historique.

De plus, l'apport de notre *pipeline* réside dans la transformation d'une image inerte en une donnée scientifique structurée. Grâce à l'exportation des détections au format JSON il devient possible un alignement précis entre les zones identifiées sur l'image et les transcriptions textuelles issues des catalogues historiques de référence comme ceux de (Adolfo Eidelstein) deviennent possible. En constituant ce jeu de données annoté de 351 images, nous contribuons à combler un peu plus le vide actuel en matière de ground truth numérique pour la sigillographie byzantine.

Enfin, nous avons placé **l'humain dans la boucle** au cur de notre processus. Notre pipeline ne cherche pas à remplacer l'expert, mais à le libérer des tâches les plus lourdes, complexes et surtout répétitives. Cette synergie Homme-Machine permet à l'historien de se concentrer sur sa véritable valeur ajoutée : La validation scientifique et l'interprétation du sens historique du

document.

En conclusion, ces résultats soulignent la viabilité de notre approche et prouvent qu'un modèle spécialisé peut devenir un outil précieux pour le déchiffrement épigraphique.

Aussi, une mise en perspective avec les travaux de Rageau et al. (2025)[13], conduits dans le cadre du projet BHAI avec une architecture similaire (YOLOv5s + ResNet18) sur un corpus de 102 images issues des collections Zacos et Tati, s'impose pour situer nos résultats. En détection, leur modèle YOLOv5s atteint une mAP@0,5 de 94,40% sur 10 plis, contre 77,76% pour notre YOLOv8s sur 5 plis. Cet écart s'explique par plusieurs facteurs structurels. D'une part, leur corpus est composé exclusivement d'images de la face épigraphique présentant un texte linéaire sans monogrammes cruciformes, là où notre corpus intègre des monogrammes entrelacés constituant un défi topologique pour les boîtes englobantes.

D'autre part, leur entraînement a bénéficié d'un GPU NVIDIA V100 16 Go, alors que nos contraintes institutionnelles nous ont limités à un GPU T4 sur Google Colab, avec des limitations mémoire significatives. Enfin, leur annotation de 29 classes exclut explicitement les 5 classes les plus rares (moins de 10 échantillons), tandis que notre corpus de 49 classes conserve l'intégralité des caractères identifiés, y compris les plus rares.

En revanche, nos résultats de classification inversent cette hiérarchie. Notre ResNet18 atteint une Top-1 accuracy de 96,66% et un F1-score macro de 93,70%, contre respectivement 92,12% et 91,65% pour Rageau et al. (2025)[13], sur un corpus trois fois plus grand (351 images contre 102) et un jeu de données de classification plus difficile (2 196 caractères sur 26 classes). Cette progression illustre l'apport de YOLOv8s dans la génération de régions d'intérêt de meilleure qualité pour l'étape de classification, ainsi que le bénéfice d'un corpus annoté plus étendu malgré l'absence de ressources GPU dédiées.

II. Classification des caractères : étude comparative

Cette section présente les résultats obtenus pour la tâche de classification des caractères byzantins extraits par le module de détection. Trois architectures ont été évaluées et comparées : un CNN entraîné depuis zéro, servant de baseline, ResNet18 et ViT-Tiny, deux modèles exploitant le transfer learning depuis ImageNet. L'évaluation porte sur un ensemble de test composé de 419 régions de caractères, réparties sur 26 classes, issues du split 80/20 appliqué au corpus de 2 196 instances. Nous analysons successivement les performances globales, la dynamique d'apprentissage à travers les courbes d'entraînement, puis les disparités de performance au niveau des classes individuelles.

1. Configuration générale

Afin de déterminer l'architecture la plus adaptée à la classification des caractères dans les monogrammes sur des sceaux byzantins, trois modèles ont été évalués sur le même split (80/20) : 1 724 images d'entraînement, 419 images de test, 26 classes.

A. Modèle 1 : CNN entraîné from scratch

Architecture : 3 blocs (Conv2D BatchNorm ReLU MaxPool) + 2 couches fully-connected. Image d'entrée : 64X64 pixels. Optimiseur : Adam (lr = 0.001). Loss : CrossEntropyLoss. Epochs : 50. Paramètres : 100% entraînés.

Époque	Train Loss	Train Acc	Test Loss	Test Acc
5	0,893	0,754	0,433	0,912
10	0,744	0,762	0,490	0,914
15	0,612	0,813	0,473	0,919
18*	**0,517**	**0,853**	**0,452**	**0,940**
20	0,511	0,859	0,430	0,940
25	0,449	0,853	0,460	0,931
30	0,433	0,877	0,457	0,938
35	0,400	0,886	0,468	0,938
40	0,390	0,880	0,466	0,938
45	0,380	0,890	0,456	0,940
50	0,374	0,892	0,456	0,940

TABLE 3 – Progression de l'entraînement du CNN (* = meilleure époque)

Meilleure accuracy test : 0.9403 (94.03%) atteinte à l'epoch 18.

B. Modèle 2 : ResNet18 (transfer learning)

Architecture : ResNet18 pré-entraîné sur ImageNet. *fine-tuning* des 2 derniers blocs résiduels + tête de classification. Image d'entrée : 224 X 224 pixels. Paramètres totaux : 11 314 522 dont 8 531 738 entraînables (75.4%). Epochs : 40.

Époque	Train Loss	Train Acc	Test Loss	Test Acc
5	0,267	0,937	0,288	0,952
10	0,215	0,948	0,364	0,948
15	0,118	0,971	0,383	0,957
20	0,097	0,971	0,417	0,957
25	0,090	0,975	0,408	0,959
30	0,077	0,976	0,416	0,962
31*	**0,068**	**0,976**	**0,397**	**0,967**
35	0,074	0,977	0,396	0,967
40	0,059	0,979	0,404	0,967

TABLE 4 – Progression de l'entraînement de ResNet18 (* = meilleure époque)

Meilleure accuracy test : 0.9666 (96.66%) atteinte à l'epoch 31.

C. Modèle 3 : ViT-Tiny

Architecture : Vision Transformer(vit_tiny_patch16_224) via la bibliothèque timm. Fine-tuning complet de l'ensemble des paramètres. Image d'entrée : 224X224 pixels. Paramètres : 5 529 434 (100% entraînés). Epochs : 40.

Époque	Train Loss	Train Acc	Test Loss	Test Acc
5	0,144	0,962	0,285	0,943
10	0,125	0,964	0,247	0,945
15	0,109	0,969	0,249	0,950
20	0,083	0,976	0,240	0,957
23*	**0,074**	**0,977**	**0,232**	**0,967**
25	0,067	0,980	0,266	0,945
30	0,058	0,977	0,252	0,962
35	0,048	0,983	0,251	0,957
40	0,042	0,985	0,252	0,964

TABLE 5 – Progression de l'entraînement de ViT-Tiny (* = meilleure époque)

Meilleure accuracy test : 0.9666 (96.66%) atteinte à l'epoch 23.

2. Résultats quantitatifs (CNN, ResNet18, ViT-Tiny)

Le tableau ci-dessous met en lumière les performances des trois architectures évaluées sur l'ensemble de test (419 images, 26 classes) de notre dataset.

Métrique	**CNN**	**ResNet18**	**ViT-Tiny**	**Meilleur**
Accuracy	94,03%	**96,66%**	96,66%	=
Précision (macro)	0,826	**0,933**	0,919	ResNet18
Rappel (macro)	0,845	**0,942**	0,918	ResNet18
F1-Score (macro)	0,832	**0,937**	0,917	ResNet18
F1-Score (pondéré)	0,935	0,964	**0,965**	ViT-Tiny
Époques	50	40	40	—
Params entraînés	100%	75,4%	100%	—

TABLE 6 – Comparaison finale des trois modèles de classification

Les résultats obtenus mettent en avant une certaine hiérarchie entre les trois architectures. Le premier modèle, un CNN simple, a été entraîné depuis zéro sur 50 epochs, a atteint une accuracy de 94,03%, avec un F1-score macro de 0,832. Ce résultat bien que louable pour une architecture comme celle-ci, cest-à-dire légère et sans pré-entraînement, révèle les limites dun modèle qui est contraint dapprendre des représentations à partir d'un corpus de taille modeste.

On remarquera que ResNet18 et ViT-Tiny ont la même accuracy 96,66%, soit un gain de 2,63% par rapport au CNN simple. Nonobstant, cest sur les métriques macro (celles qui traitent toutes les classes pareillement sans tenir compte de leur fréquence) que ResNet18 se distingue. Ce dernier obtient un F1-score macro de 0,937ă, contre 0,917ă pour le ViT-Tiny. Cet écart qu'on pourrait qualifier de minime en apparence, traduit la capacité de ce modèle à maintenir des performances équilibrées sur les classes dites rares, ce qui constitue un enjeu majeur dans notre corpus à forte hétérogénéité.

Il est intéressant de noter que ResNet18 atteint ce résultat en ne sappuyant que sur une partie de ses paramètres. Autrement dit, ResNet18 mobilise un fine-tuning partiel (75,4% des paramètres entraînés, soit environ 8,5M sur 11,3M), alors que ViT-Tiny entraîne l'intégralité de ses 5,5M de paramètres. Ainsi en termes defficacité paramétrique, ResNet18 présente un rapport performance/coût dentraînement plus favorable pour un corpus comme le nôtre.

3. Courbes d'apprentissage

L'analyse qu'on peut faire des courbes dapprentissage (cf. Figure 18) nous permettra sans nul doute de caractériser le comportement plus ou moins dynamique de chaque architecture au

fil des epochs. Le CNN simple présente une convergence lente et progressive sur 50 epochs. La loss de test se stabilise autour de 0,45 dès l'époque 10, sans une amélioration notable par la suite, tandis que l'accuracy plafonne à 94% avec de légères oscillations. Ce comportement indique que le modèle exploite rapidement l'information disponible sans pouvoir extraire des représentations suffisamment discriminantes au-delà d'un certain seuil.

ResNet18, quant à lui, converge beaucoup plus rapidement, ce qui est sans doute dû au transfert de connaissance depuis ImageNet. Dès les 5 premières époques, la test accuracy dépasse 95%, et le modèle atteint sa meilleure performance à l'epochs 31 (96,66%). La loss de test descend à des valeurs proches de 0,39 et reste stable, témoignant d'une généralisation solide sans *overfitting* (surapprentissage).

Le ViT-Tiny affiche quant à lui la loss de test la plus basse des trois (autour de 0,230,26), mais sa courbe d'accuracy présente davantage de variabilité entre les epochs. Le modèle atteint sa meilleure performance (96,66%) à l'époque 23, puis oscille légèrement sans progression nette. Ce comportement relatif s'explique par la nature même des Vision Transformers. ViT découpe limage en petits patches quil va traiter comme des séquences de mots, utilisant ainsi des mécanismes d'attention globale plutôt que des filtres locaux. Tout ceci pour dire que les Vision Transformers ont besoin de beaucoup de données pour mieux généraliser. Sur un corpus de 2 196 images, le ViT-Tiny opère en limite de sa zone de confort.

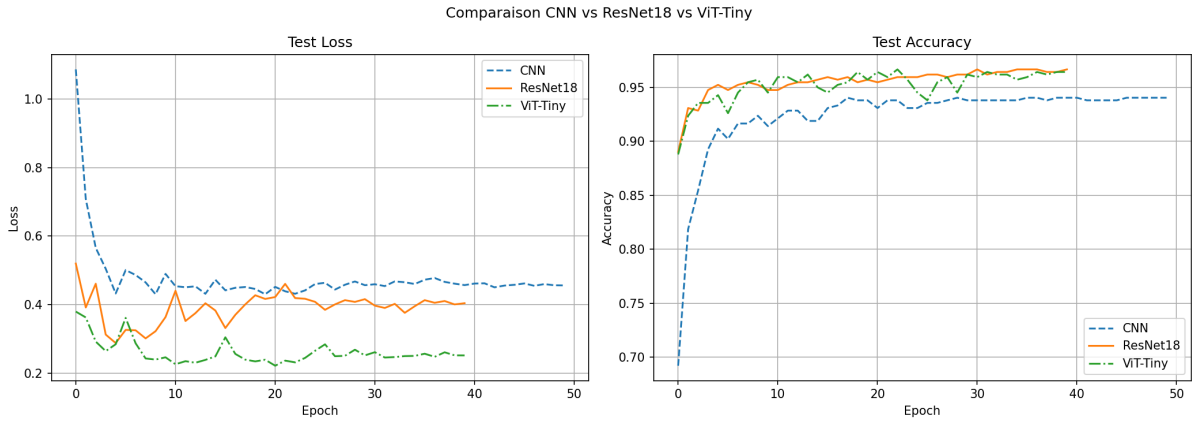


FIGURE 20 – courbes Test Loss et Test Accuracy CNN vs ResNet18 vs ViT-Tiny

4. Analyse par classe et cas problématiques



FIGURE 21 – F1-Score par classe pour les 26 classes (3 modèles)

L'analyse du F1-score par classe (cf. Figure 21) révèle une performance globalement homogène de ResNet18, avec néanmoins des disparités propres à la structure du corpus.

La majorité des classes atteint un F1-score supérieur à 0,93 pour ResNet18. Les caractères les mieux reconnus sont ceux dotés d'une morphologie distinctive et d'un nombre suffisant d'exemples : $\Phi, T, H, B, \Theta, I, O, \Pi, M$ atteignent un F1 de 1,0 sur les trois modèles. La forme géométrique saillante de ces caractères (cercles fermés, barres horizontales, symétries marquées) facilite la séparation des classes même dans des conditions de dégradation partielle.

Trois classes concentrent particulièrement notre attention :

Classe	Support	CNN F1	ResNet18 F1	ViT-Tiny F1	Motif
(sigma)	3	**0,000**	**0,000**	0,400	Rareté
(lambda)	13	0,828	0,828	0,929	Ambiguïté morphologique
(psi)	1	**0,000**	**1,000**	**0,000**	Rareté extrême

TABLE 7 – Classes présentant un F1-score inférieur à 0,93 et facteurs explicatifs

(sigma) est la classe la plus problématique : F1 = 0,000 pour le CNN et ResNet18, et seulement 0,400 pour le ViT-Tiny. Avec un support de seulement 3 instances dans l'ensemble de test, cette classe illustre de manière exemplaire l'effet de la rareté sur les métriques de classification. Le modèle ne dispose pas d'exemples suffisants pour apprendre une frontière de décision fiable, et les erreurs de classification sont inévitables statistiquement.

(lambda) présente un F1 de 0,828 pour le CNN et ResNet18, légèrement supérieur (0,929) pour le ViT-Tiny. Sa forme triangulaire ouverte peut être confondue avec (alpha) ou (delta) selon le niveau de lisibilité des caractères du sceau, ce qui explique une précision modérée (0,75 pour ResNet18 notamment).

(psi) constitue quant à lui un cas particulièrement révélateur. Avec un support d'une seule instance dans l'ensemble de test, ce caractère obtient un F1 = 0,000 pour le CNN et le ViT-Tiny, tandis que ResNet18 parvient à le classifier correctement avec un F1 de 1,000. Ce résultat, à première vue paradoxal, s'explique par la capacité de ResNet18 à mobiliser des représentations préapprises suffisamment robustes pour reconnaître un caractère qu'il n'a presque jamais rencontré. Il illustre également les limites statistiques d'une évaluation sur des classes à support minimal : un seul exemple correct ou incorrect suffit à faire basculer le F1 de 0 à 1, ce qui invite à interpréter ces scores avec prudence.

Il est important de souligner que ces difficultés ne remettent pas en cause la robustesse globale du pipeline. En effet, le diagramme en barres par classe (cf. Figure X) confirme que la majorité des 26 classes sont bien classifiées par les trois modèles (F1 > 0,90). Les écarts apparaissent principalement sur les classes à faible effectif : les classes problématiques correspondent systématiquement à des classes sous-représentées (support 3) ou à des caractères dont la morphologie présente une ambiguïté intrinsèque. Ainsi, (support = 3) n'est correctement classifié par aucun modèle CNN ou ResNet18 (F1 = 0,000), tandis que ViT-Tiny obtient un F1 de 0,400. À l'inverse, ResNet18 est le seul à classifier correctement (support = 1). Ces limitations sont structurellement liées à la nature même du corpus sigillographique et non à un défaut de la méthode d'apprentissage, ce qui confirme que l'augmentation du corpus pour les classes rares constitue une priorité.

III. Perspectives d'amélioration

Bien que les résultats obtenus par notre pipeline soient encourageants, tant pour la détection avec YOLOv8s que pour la classification avec le ResNet18, ce travail ouvre plusieurs pistes d'amélioration pour rendre l'outil encore plus performant et plus robuste face à la complexité des monogrammes byzantins.

En premier lieu, la priorité absolue réside dans l'augmentation et l'équilibrage du corpus comme nous l'avons d'ailleurs évoqué lors de l'analyse des classes ń rebelles ż comme le sigma ou le Psi, le modèle ne peut apprendre efficacement sans une base d'exemples suffisamment représentative. Pour pallier donc la rareté de certains caractères, nous pourrions explorer des techniques d'augmentation de données pour cela nous devons mener des campagnes d'annotations ciblées afin d'intégrer de nouveaux exemples de monogrammes et de caractères authentiques. A cet égard, l'exploitation des outils réels par des outils d'augmentation intégrés à supervisely

nous permettrait de multiplier les variantes de nos images réelles par des transformations géométriques. Cela aiderait le modèle à mieux appréhender la nature tridimensionnelle du relief et les déformations du métal, qui modifient la perception des traits constitutifs du monogramme sans pour autant dénaturer la source historique.

Ensuite, une perspective technique majeure que nous avons identifiée concerne l'évolution vers la segmentation d'instance pour optimiser la gestion des superpositions propres aux monogrammes. Dans ces structures, notamment cruciformes, les lettres ne sont pas simplement juxtaposées mais s'entremêlent souvent sur une même barre verticale ou horizontale, ce qui rend la délimitation par boîte englobante (*bounding box*) intrinsèquement imprécise. En exploitant les annotations polygonales que nous avons déjà réalisées, nous pourrions entraîner des modèles capables d'isoler la silhouette exacte de chaque lettre au pixel près. Cela permettrait de distinguer deux caractères fusionnés là où le système actuel ne voit parfois qu'une seule forme globale, améliorant notamment la détéction des barres verticales fines qui servent de support commun à plusieurs lettres.

Par ailleurs, nous envisageons de renforcer la synergie « Homme-Machine » en développant une interface de validation ergonomique fondée sur le principe de l'apprentissage actif que nous appelons (active learning) . L'idée est de permettre au sigillographe de corriger les détections au sein des monogrammes en un clic afin que ces corrections servent immédiatement à ré-entraîner le modèle. Dans cette optique, le modèle deviendrait de plus en plus performant au fur et à mesure de son utilisation, le chercheur pouvant focaliser son expertise sur les cas où le score de confiance est faible ou les ligatures particulièrement ambiguës, réduisant ainsi progressivement le temps d'intervention humaine sans jamais sacrifier la rigueur scientifique.

Enfin, une ouverture passionnante consisterait à coupler la reconnaissance visuelle à un modèle de langue spécialisé en grec médiéval. En intégrant une couche de probabilité linguistique basée sur les titulatures et les formules d'invocation classique, le système pourrait littéralement « deviner » un caractère partiellement effacé au sein du monogramme en s'appuyant sur le contexte des autres lettres identifiées. Ce passage de la simple forme au sens permettrait d'automatiser l'alignement avec les transcriptions de référence issues des catalogues historiques, marquant une étape décisive vers un déchiffrement totalement assisté des monogrammes les plus complexes. En somme, ce mémoire pose les jalons d'un outil perfectible mais déjà opérationnel, dont les améliorations futures devront tendre vers une plus grande autonomie tout en gardant à l'esprit que l'expertise humaine reste le pilier indispensable pour transformer une détection algorithmique en une connaissance historique fiable.

Conclusion Générale

Ce mémoire s'inscrit également dans le cadre du projet BHAI (Byzantine Hybrid Artificial Intelligence), dont l'ambition est de concevoir un pipeline semi-automatique capable de détecter et de classifier les caractères présents sur les monogrammes de sceaux byzantins. Face à la richesse et à la complexité de ce patrimoine dautres temps, l'enjeu était de proposer une approche computationnelle rigoureuse, capable de traiter des données visuelles historiques hétérogènes.

La tâche s'est articulée autour de deux volets complémentaires. La première, consacrée à la détection et cur de ce travail de recherche, a reposé sur l'entraînement d'un modèle YOLO sur un corpus de 351 images de sceaux byzantins, réparties en 49 classes de caractères, selon un protocole de validation croisée à 5 plis. Les résultats obtenus avec un mAP@50 moyen de 0.778 et un mAP@50 de 0.855 sur le meilleur fold confirment la viabilité de cette architecture pour la détection de caractères sur des sources historiques dégradées. Le second axe (axe plus expérimental et proposé par le binôme détudiants), dédié à la classification, a permis de comparer trois architectures : un CNN entraîné *from scratch*, ResNet18 et ViT-Tiny sur un corpus de 2 196 caractères extraits répartis en 26 classes. ResNet18 s'est imposé comme le modèle le plus adapté, avec une accuracy de 96.66%, alliant performance et stabilité d'entraînement.

Au-delà des résultats techniques, ce travail a permis de mettre en lumière une problématique fondamentale : la qualité variable des données sources. L'état de conservation des sceaux (tantôt lisibles, tantôt corrodés ou fragmentaires) introduit une hétérogénéité qui affecte directement les performances des modèles. C'est précisément cette observation qui a motivé une réflexion sur la restructuration du travail d'annotation. Ces nouvelles annotations (basées sur des accords inter-annotateurs) devront définir un schéma de qualification en quatre niveaux (M0 à M3) appliqué à un corpus dimages élargi issu de quatre collections distinctes[5]. Ce schéma constituera, nous le croyons, une contribution méthodologique nouvelle : en transformant la qualité visuelle d'un sceau en donnée exploitable, il ouvrira la voie à un filtrage automatique en amont du pipeline, susceptible d'améliorer significativement la robustesse et la précision des étapes de détection et de classification.

Cette perspective rejoint une démarche plus large de *datafication* des sources, dans laquelle la qualité n'est plus subie comme un bruit, mais modélisée comme une information à part entière. Les travaux futurs pourront s'appuyer sur ce pipeline pour entraîner des modèles plus robustes, mieux pondérés selon la fiabilité des images d'entrée. Une prise en compte complète de ce point constituerait une avancée significative, transformant un système de détection-classification en un pipeline pleinement conscient de la qualité de ses données.

Ce mémoire représente ainsi une étape dans un projet plus vaste, dont les fondations (corpus, pipeline, protocole d'évaluation) sont désormais posées. Il illustre comment les humanités numériques, à l'intersection de la sigillographie, de la paléographie et de l'apprentissage au-

5. Collections Adolfo Eidelstein, Zacos, Seyrig et Tatish, annotées manuellement depuis la plateforme Supervisely dans le cadre de ce mémoire.

tomatique, peuvent renouveler notre rapport aux sources du passé, non pas en les simplifiant, mais en en respectant la complexité.

Annexe A

Exemples dimages annotées

Vous trouverez ici des exemples de monogrammes annotés manuellement par notre équipe sur la plateforme Supervisely. Ces captures donnent une idée du caractère chronophage de cette entreprise : l'annotation de l'ensemble du corpus de 351 images nous a mobilisés pendant trois semaines.

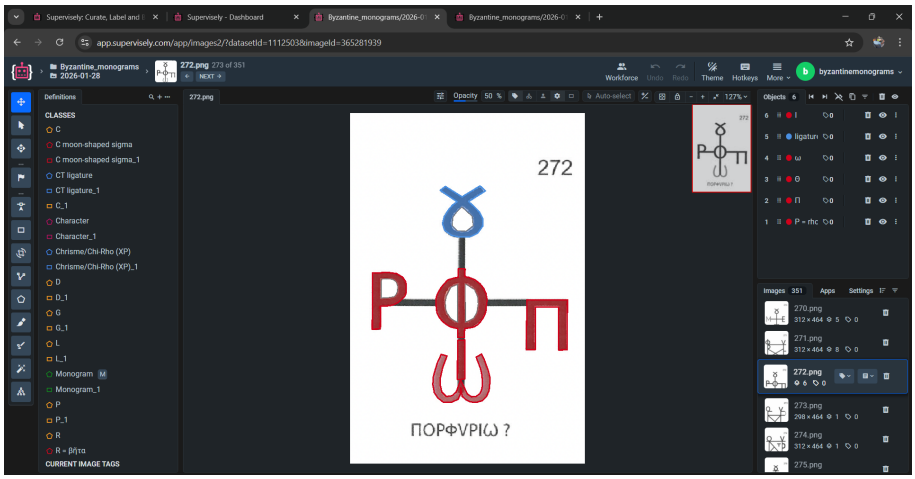


FIGURE 22 – Monogramme annoté 268 (Supervisely)

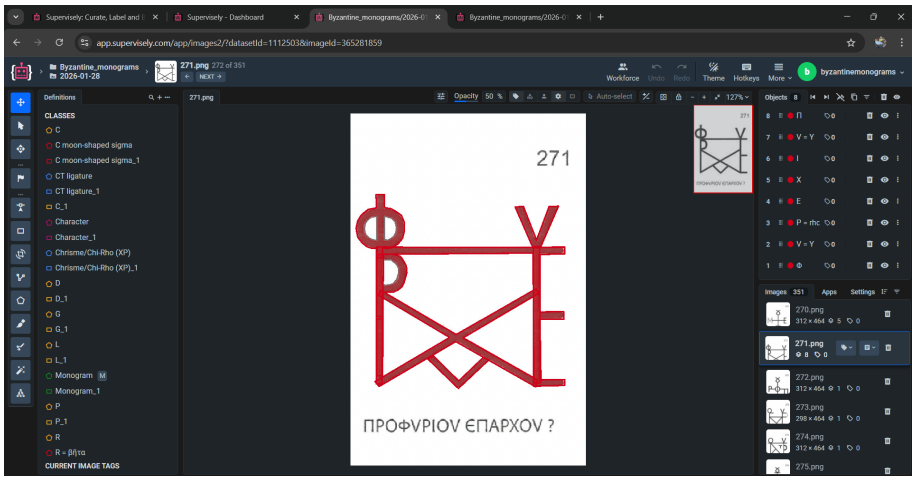


FIGURE 23 – Monogramme annoté 270 ? (Supervisely)

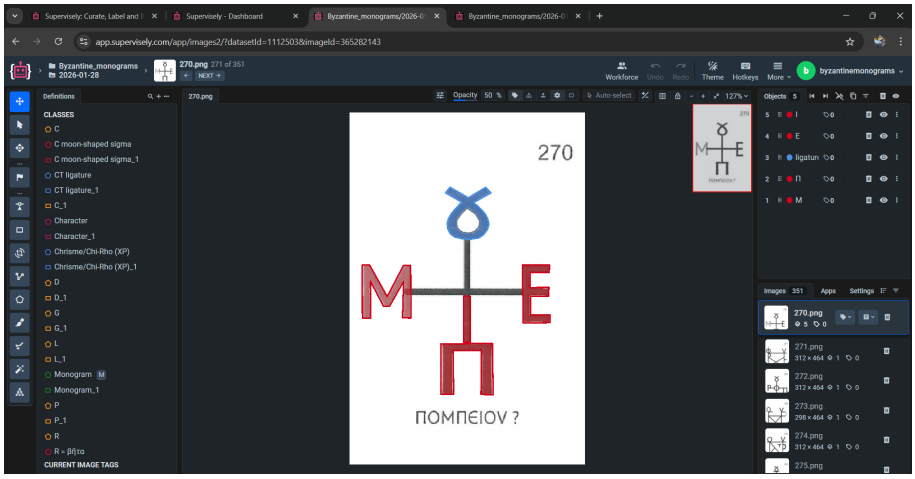


FIGURE 24 – Monogramme annoté 271 ? (Supervisely)

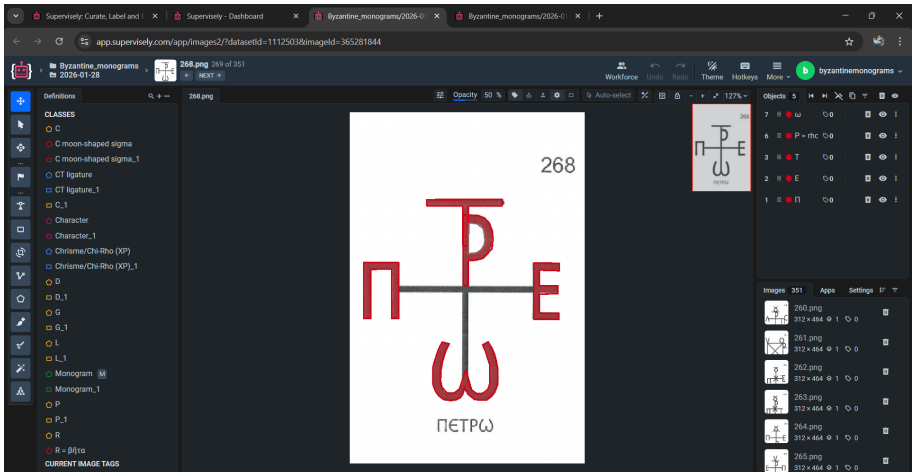


FIGURE 25 – Monogramme annoté 272 (Supervisely)

Annexe B

Tableau des classes de caractères

B.1 Classes de détection YOLOv8s (49 classes)

TABLE 8 – Liste des 49 classes utilisées pour la détection

ID	Classe	ID	Classe
0	(alpha)	25	S
1	C	26	G
2	P	27	L
3	(beta)	28	R
4	(sigma)	29	U
5	Ligature OU	30	V
6	C moon-shaped sigma	31	(tau)
7	CT ligature	32	(upsilon)
8	Monogram	33	(phi)
9	R =	34	(chi)
10	(gamma)	35	(psi)
11	(delta)	36	(oméga)
12	(epsilon)	37	V = Y
13	(zeta)	38	Comma
14	(eta)	39	Chrisme/Chi-Rho ()
15	= rho	40	S =
16	(mu)	41	=
17	(nu)	42	= double
18	(xi)	43	(thêta)
19	(omicron)	44	(iota)
20	(pi)	45	(kappa)
21	trait ligature sup	46	(lambda)
22	coronis	47	Character
23	= sigma	48	sss
24	D		

B.2 Classes de classification (26 classes)

On notera que le nombre de classes diffère entre la phase de détection (49 classes) et la phase de classification (26 classes). Cette réduction s'explique par un travail de nettoyage du

jeu de données effectué en amont de la classification. Plusieurs critères ont motivé ce choix :

— Les classes contenant un nombre insuffisant d'exemples ont été écartées afin de garantir un volume d'entraînement minimal par classe.

— Certaines classes spécifiques à la détection comme *Monogram*, *Character*, *sss*, *coronis* ou *trait ligature sup* correspondent à des entités graphiques globales ou des artefacts d'annotation qui ne constituent pas des caractères à proprement parler.

— Les deux tâches ne poursuivent pas le même objectif : la localisation est moins sensible au déséquilibre des classes que la classification, qui demande un nombre conséquent d'exemples par classe pour distinguer des morphologies proches.

Ce nettoyage a permis de passer de 49 à 26 classes, garantissant ainsi un apprentissage fiable et des métriques d'évaluation interprétables.

ID	**Classe**	**ID**	**Classe**	**ID**	**Classe**
0	V	9	(nu)	18	(rho)
1	Ligature OU	10	(xi)	19	(sigma)
2	(alpha)	11	(omicron)	20	(tau)
3	(beta)	12	(pi)	21	(phi)
4	(gamma)	13	(kappa)	22	(chi)
5	(delta)	14	(lambda)	23	(psi)
6	(epsilon)	15	(mu)	24	(oméga)
7	(zeta)	16	(eta)	25	(sigma lunaire)
8	(thêta)	17	(iota)		

TABLE 9 – Liste des 26 classes retenues pour la classification après nettoyage du corpus

Bibliographie

[1] Yannis Assael, Thea Sommerschield, Brendan Shillingford, Mahyar Bordbar, John Pavlopoulos, Marita Chatzipanagiotou, Ion Androutsopoulos, Jonathan Prag, and Nando de Freitas. Restoring and attributing ancient texts using deep neural networks. *Nature*, 603(7900):280–283, 2022. (Cité en page 25)

[2] Frédéric Barbier. *L'Empire du livre : le livre imprimé et la construction de l'Allemagne contemporaine*. Éditions du Cerf, Paris, 1995. (Cité en page 16)

[3] Rui Cabral, Maria De Iorio, and Andrew Harris. From coin to data : the impact of object detection on digital numismatics. *Digital Scholarship in the Humanities*, 40(3):733–746, 2025. (Cité en page 24)

[4] Jean-Claude Cheynet. *La société byzantine : l'apport des sceaux*. Association des amis du Centre d'histoire et civilisation de Byzance, Paris, 2008. (Cité en page 16)

[5] Victoria Eyharabide. Artificial intelligence applied to byzantine sigillography : Current research, challenges, and future perspectives. *Digital Medievalist*, 17(1):1–16, 2024. In Digital Approaches to Medieval Sigillography, ed. Martina Filosa, Claes Neuefeind, and Claudia Sode, special issue. (Cité en pages 11 et 25)

[6] Victoria Eyharabide, Laurence Likforman-Sulem, Luca Orlandi, et al. Study of historical byzantine seal images : the BHAI project for computer-based sigillography. In *Proceedings of the 7th International Workshop on Historical Document Imaging and Processing (HIP '23)*, pages 49–54, New York, NY, USA, 2023. Association for Computing Machinery. (Cité en pages 12 et 25)

[7] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 580–587, 2014. (Cité en page 22)

[8] Glenn Jocher et al. Ultralytics YOLOv5 v6.0. https://doi.org/10.5281/zenodo.5563715, 2021. (Cité en page 23)

[9] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017. (Cité en page 22)

[10] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD : Single shot MultiBox detector. In *Computer Vision – ECCV 2016*, volume 9905 of *Lecture Notes in Computer Science*, pages 21–37, Cham, 2016. Springer. (Cité en page 22)

[11] Tom Malzbender, Dan Gelb, and Hans Wolters. Polynomial texture maps. In *Proceedings of SIGGRAPH 2001*, pages 519–528, New York, 2001. ACM Press. (Cité en page 17)

[12] John Nesbitt and Nicolas Oikonomidès. *Catalogue of Byzantine Seals at Dumbarton Oaks and in the Fogg Museum of Art*. Dumbarton Oaks Research Library and Collection, Washington D.C., 1991–2009. (Cité en page 21)

[13] T. Rageau, L. Likforman-Sulem, A. Fiandrotti, V. Eyharabide, B. Caseau, and J. C. Cheynet. Character recognition in Byzantine seals with deep neural networks. *Digital Applications in Archaeology and Cultural Heritage*, 37:e00403, 2025. (Cité en pages 12 et 52)

[14] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once : Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. (Cité en pages 22 et 28)

[15] Joseph Redmon and Ali Farhadi. YOLOv3 : An incremental improvement. *arXiv*, 2018. arXiv:1804.02767. (Cité en page 23)

[16] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN : Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, pages 91–99, 2015. (Cité en page 22)

[17] Rosalina Rosalina. Automated detection of ancient indonesian coins for historical numismatic investigations. *Journal of Informatics and Telecommunication Engineering (JITE)*, 8(1), 2024. (Cité en page 24)

[18] Gustave Schlumberger. *Sigillographie de l'Empire byzantin*. Ernest Leroux, Paris, 1884. (Cité en page 11)

[19] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. (Cité en page 19)

[20] Zhengxia Zou, Keyan Chen, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years : A survey. *Proceedings of the IEEE*, 111(3):257–276, 2023. (Cité en page 21)